

**OPTIMIZING HINDI-TO-ENGLISH TRANSLATION:
ADAPTING mBART FOR RESOURCE EFFICIENCY**

Report submitted in partial fulfilment of the requirements for the

B. Tech. degree

in

Computer Science with specialization in Big Data Analytics

By

DAKSH KALUCHA 2021UCB6048

PRAKHAR VERMA 2021UCB6047

SACHIT GUPTA 2021UCB8015

Under the supervision of

Dr. Bharti Nagpal

Assistant Professor



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
NETAJI SUBHAS UNIVERSITY OF TECHNOLOGY (NSUT)
GEETA COLONY, NEW DELHI**

May 2025

DECLARATION BY THE STUDENTS

We, the undersigned, hereby declare that the work entitled “OPTIMIZING HINDI-TO-ENGLISH TRANSLATION: ADAPTING mBART FOR RESOURCE EFFICIENCY” is an original contribution carried out by us, Daksh Kalucha, Prakhar Verma, and Sachit Gupta, as part of our academic requirements. We confirm that this project is a result of our independent efforts and research under the guidance and supervision of Dr. Bharti Nagpal. We have duly acknowledged all sources of information used during the course of the project and affirm that this work is free from major plagiarism. We take responsibility for all content’s originality in developing on completing this work and are mindful of the academic ethical policies regarding research and project submissions.

AI Usage Declaration for Bachelor Thesis Project Report

Report Title: Optimizing Hindi-to-English Translation: Adapting mBART for Resource Efficiency

Student Name: Daksh Kalucha, Prakhar Verma, Sachit Gupta

Student ID: daksh.ug21@nsut.ac.in, prakhar.verma.ug21@nsut.ac.in, sachit.gupta.ug21@nsut.ac.in,

Degree Program: B. Tech. in Computer Science with Specialization in Big Data Analytics

University: Netaji Subhas University of Technology, Delhi

Supervisor: Dr. Bharti Nagpal

We, Daksh Kalucha, Prakhar Verma and Sachit Gupta, declare the use of Artificial Intelligence (AI) tools in this report is inline with Netaji Subhas University of Technology, Delhi and Supervisor guidelines.

We confirm that we did **not** use AI for prohibited activities, including generating substantial portions of the report, submitting AI text as original work without significant revision and human overview, or submitting unreviewed AI-translated text.

This report reflects our team work, critical analysis, understanding, and application in problem- solving, evaluation, and novel creation.

Details of AI Tool Usage:

AI Tool Used (Specify name and version, if possible)	Purpose of Use	Stage of Report Generation Process (e.g., Brainstorming, Drafting Section X, Editing)
ChatGPT 4	Generating ideas for the chapter “Discussion”.	Initial Brainstorming Phase
ChatGPT 4	Generating ideas for the chapter “Literature Review”	Initial Brainstorming Phase

We take full responsibility for the content and integrity of this report.

Student’s Signature:

Date: 20/05/2025

CERTIFICATE

This is to certify that the project titled “Optimizing Hindi-to-English Translation: Adapting mBART for Resource Efficiency” has been successfully completed by Daksh Kalucha, Prakhar Verma, and Sachit Gupta under the guidance and supervision of Dr. Bharti Nagpal, in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology (B.Tech.) in Computer Science and Engineering, with specialization in Big Data Analytics (CSDA branch), at Netaji Subhas University of Technology, East Campus, Geeta Colony, Delhi.

Dr. Bharti Nagpal
Assistant Professor
Department of CSE, NSUT

Prof. Vishal Bhatnagar
Head of Department
Department of CSE, NSUT

TABLE OF CONTENTS

List of Figures	vii
List of Tables	viii
List of Abbreviations	x
Acknowledgement	xiii
Abstract	xiv
Chapter 1- Introduction	1-11
1.1 Background and Motivation	1
1.2 Literature Review	6
Chapter 2- Methodology	12-20
2.1 Data Collection	13
2.2 Data Pre-processing	15
2.3 Model Preparation	16
2.4 Evaluation Metrics Used	18
2.5 Platforms and Frameworks Used	20
Chapter 3- Results	21-22
Chapter 4- Discussion	23-50
4.1 Comparing performance of LoRA with full fine-tuning	23
4.2 Comparing performance of QLoRA with full fine-tuning	23
4.3 Base Model Performance and why it needs Fine-tuning	24
4.4 Optimization Strategies for LoRA	30
4.5 Optimizing QLoRA	42
4.6 Comparative analysis with similar work	47

Conclusion and Future work	51
References	54
Appendices	59-65
Appendix A- Paper Acceptance Mail	59
Appendix B- Certificate of Presentation and Paper Publishing Status	60
Appendix C- Conference Paper Work & Extra Work Done	62
Appendix D- Plagiarism Report	64

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1	Research methodology for refining translation model	12
2	Reparameterization performed in LoRA technique.	17
3	QLoRA compresses the model, refines the model using LoRA and uses the concept of paging to handle memory spikes, providing better memory efficiency than LoRA.	18
4	GPU Memory and Training Time Comparison Across Fine-Tuning Methods	24
5	Inference Speed and Translation Quality Scores (METEOR and ChrF) Across Fine-Tuning Approaches	24
6	Performance and Inference Time Comparison between Base Model and Full Fine-Tuned Model	29
7	GPU Memory Usage Comparison between Base Model and Full Fine-Tuned Model	29
8	Inference Time Comparison Across LoRA Ranks	31
9	GPU Memory Usage Across LoRA Ranks	31
10	Impact of Alpha Values on Translation Quality: Relationship Between LoRA Alpha Parameter and METEOR/ChrF Scores in Hindi-English Translation	35
11	Evaluation Scores Across Dropout Rates	36
12	Training Time per Epoch vs Dropout Rate	36
13	Comparison of METEOR and ChrF scores across different LoRA target module configurations.	41
14	Average inference time (in seconds) across different LoRA target module configurations.	41
15	Average Training Time Per Epoch (in sec) Across Various Hyperparameters Experimented for QLoRA	46
1C	Architecture depicting the discriminative fine-tuning model	63

LIST OF TABLES

Table No.	Table Name	Page No.
1	Summary of Key Literature on PEFT and Machine Translation	8
2	Sentence Distribution by Primary Categories and Tenses	14
3	Standard Sentences and Idiomatic Expressions Breakdown	14
4	Voice and Speech Constructions Breakdown	15
5	Common data preprocessing parameters applied to the Hindi-English dataset across all experimental configurations	16
6	Common training hyperparameters for Hindi-English mBART translation model.	16
7	Performance evaluation metrics and computational resource measurements tracked across model variants	18
8	Comparative Performance Analysis of Fine-Tuning Approaches: Full, LoRA, and QLoRA	21
9	Translation response improvement seen from base model to fully fine-tuned model	25
10	Comparative Power and Resource Utilization of the Base Model and Full Fine-Tuned Model	26
11	Comparison of translation response of base model and LoRA fine-tuned model	26
12	Comparison of translation response of base model and QLoRA fine-tuned model	28
13	Impact of LoRA Rank on Model Performance, Resource Utilization, and Trainable Parameters	30
14	Impact of Alpha Values on Model Performance and Resource Utilization	32
15	Translation response improvement seen across models fine-tuned using different alpha values	32
16	Effect of Dropout on Training and Evaluation Metrics	36

17	Comparative analysis of LoRA fine-tuning configurations across different attention module combinations	37
18	Translation response seen across models fine-tuned using different target modules used	38
19	Performance Analysis of Model Training with Different Bit Quantization Settings	42
20	Memory and Performance Metrics Comparison for Double Quantization Configurations	43
21	Evaluation of Training and Inference Performance Across Computation Data Types	44
22	Comparison of Memory Usage and Training Performance Metrics between Paged and Non-Paged Optimization	45
23	Comparison with Other Similar Works	48

LIST OF ABBREVIATIONS

- **AI** – Artificial Intelligence: The simulation of human intelligence in machines that are programmed to think and learn.
- **MT** – Machine Translation: Automated translation of text from one language to another using computer systems.
- **NMT** – Neural Machine Translation: A type of MT that uses neural networks to predict and generate translations.
- **SMT** – Statistical Machine Translation: An older MT approach that uses statistical models based on bilingual text corpora.
- **DLM** – Deep Learning Model: A machine learning model with multiple layers that processes data through artificial neural networks.
- **GPT** – Generative Pre-trained Transformer: A language model trained on large datasets to generate human-like text.
- **LLM** – Large Language Model: A language model with billions of parameters, capable of understanding and generating complex text.
- **ML** – Machine Learning: A subset of AI that enables systems to learn patterns from data and improve over time.
- **DL** – Deep Learning: A specialized ML approach involving deep neural networks to model complex data patterns.
- **EU** – European Union: A political and economic union whose data privacy regulations (e.g., GDPR) impact AI and MT systems.
- **LoRA** – Low-Rank Adaptation: A technique for efficient fine-tuning of large models by updating only low-rank matrices.
- **QLoRA** – Quantized Low-Rank Adaptation: An enhanced LoRA method using quantization to reduce memory usage during fine-tuning.
- **RNN** – Recurrent Neural Network: A type of neural network used for sequential data where outputs depend on previous computations.
- **LSTM** – Long Short-Term Memory: A specialized RNN that can capture long-range dependencies in sequence data.

- **HBM**T – Hybrid Machine Translation: Combines rule-based, statistical, and/or neural methods for improved translation quality.
- **PEFT** – Parameter-Efficient Fine-Tuning: A technique that fine-tunes only a subset of model parameters to save resources.
- **SOV** – Subject-Object-Verb: A syntactic sentence structure where the subject precedes the object and the verb (e.g., Hindi).
- **SVO** – Subject-Verb-Object: A syntactic sentence structure where the subject comes first, followed by the verb and object (e.g., English).
- **BLEU** – Bilingual Evaluation Understudy: A metric to evaluate the quality of machine-translated text against human references.
- **TM** – Translation Memory: A database of previously translated segments used to assist and improve future translations.
- **AIGC** – AI-Generated Content: Content such as text, images, or audio created by artificial intelligence systems.
- **MTLoRA** – Multi-task Low-Rank Adaptation: A LoRA method adapted for multi-task learning across various NLP tasks.
- **METEOR** – Metric for Evaluation of Translation with Explicit ORdering: A translation evaluation metric that aligns words and considers synonyms and paraphrasing.
- **ChrF** – Character F-score: A metric based on character n-gram precision and recall to evaluate translation quality.
- **MB** – Megabytes: A unit of data storage equal to approximately one million bytes, used to measure file or memory size.
- **ROUGE-L** – Recall-Oriented Understudy for Gisting Evaluation – Longest: A metric evaluating the longest common subsequence between machine-generated and reference texts.
- **LCS** – Longest Common Subsequence: A sequence that appears in the same order within two texts used in similarity measurement.
- **CSV** – Comma-Separated Values: A plain text file format that stores tabular data with each field separated by a comma.
- **CUDA** – Compute Unified Device Architecture: NVIDIA's parallel computing platform and API for using GPUs for general computing.

- **nlk** – Natural Language Toolkit: A Python library for working with human language data in NLP tasks.
- **GPU** – Graphics Processing Unit: A hardware processor optimized for rendering graphics and accelerating AI computations.
- **GB** – Gigabytes: A unit of data equal to about one billion bytes, commonly used to quantify storage or memory size.
- **I/O** – Input/Output Operations: Data transfer processes between the computer's memory and external devices or storage.
- **OOM** – Out of Memory: An error state where a system or process exceeds available memory capacity.
- **NF4** – Normalized Float 4-bit: A 4-bit floating-point format used for quantization in model compression to reduce memory.
- **FP4** – Floating Point 4-bit: Another 4-bit floating-point format, optimized for efficient storage and computation in low-resource settings.

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Bharti Nagpal, under whose invaluable guidance and constant support this project, "OPTIMIZING HINDI-TO-ENGLISH TRANSLATION: ADAPTING mBART FOR RESOURCE EFFICIENCY" was successfully completed. Her expertise, insights, and feedback throughout this research have helped us tremendously. We are also deeply grateful to the faculty and staff of the Computer Science and Engineering Department, NSUT, East Campus, Delhi, for providing us with the necessary resources and a conducive environment to pursue this project. We also thank Dr. Shobha Bhatt in reviewing our work and providing crucial feedback.

ABSTRACT

With the increasing demand for efficient task-specific adaptation of large models in computationally constrained environments, the research explores methods to optimize translation quality while minimizing computational overhead. This work focuses on addressing the challenges of Hindi-to-English machine translation (MT) with a focus on parameter-efficient fine-tuning (PEFT) techniques, namely Low-Rank Adaptation (LoRA) and Quantized LoRA (QLoRA), applied to the pre-trained mBART model. Low-Rank Adaptation trains the layer’s parameters which are stores as low-rank decomposition. QLoRA performs quantization of the paramters first, then train the model’s layers using LoRA. Utilizing the pre-trained mBART model as a foundation, fine-tuned it with a curated dataset comprising 9,900 diverse Hindi-English sentence pairs, categorized across linguistic dimensions such as idiomatic expressions, tenses, and grammatical structures. The experiments reveal that LoRA achieves a 66.12% reduction in GPU memory usage, a 70.01% reduction in training time compared to full fine-tuning, and 8.16% reduction in inference time, while maintaining comparable METEOR and ChrF scores. QLoRA, leveraging 4-bit quantization and innovative memory optimizations, demonstrated a 90.06% reduction in GPU memory usage. These results underline the potential of PEFT techniques, applied to pre-trained multilingual models like mBART, in creating scalable, efficient MT systems capable of running on resource-constrained hardware.

CHAPTER-1

INTRODUCTION

MT is where computer-based systems are used to convert text from one language to another. These systems can function entirely independently (fully automated) or in combination with human assistance (such as post-editing or pre-editing by human translators) [1]. Increasingly, there is more and more information being produced that makes it impossible to meet the demand through having subject matter experts. Which means there is an increasing demand for information access tools for various audiences. Its emergence as a potential solution to this problem has made it a hot topic in both research and professional settings [2]. Real-time translation in online communication systems, cross-language business communication [3], cross-lingual retrieval [4], and supporting clinical communication [5] by eliminating the language gap between healthcare providers and patients are just a few examples of what it could do.

1.1 Background and Motivation

With developments in Artificial Intelligence (AI), MT has become better than in the past decades. Improves translation accuracy, fluency, and contextual understanding. Take Neural MT (NMT) for example, it relies on models such as transformers that facilitate the discovery of complex linguistic correlations, using attention mechanisms to focus on the most important parts of the source text to achieve more natural and accurate translations. Statistical MT (SMT) approaches to translation use statistical models created from bilingual corpora, and method, such as tokenization is crucial in comprehension and processing human writing. Additionally, fuzzy logic algorithms remove ambiguity in translations, while feature extraction methods remove non-useful linguistic information from the source texts. Together, all these improvements provide an ever-more reliable and scalable AI-driven MT solution in a world where the demand for effective multilingual communication keeps growing [6].

So, the methods discussed before are pre-training methods where the layers of a model are new i.e. untrained. This method needs a huge amount of data, which can be a slight constraint for building a machine translation system. As a means to counteract this issue, research has resorted to employing transfer learning techniques. Enhancing models with

existing knowledge through transfer learning helps them to work with only small amounts of data by supporting that knowledge learned by a model trained on a more general task applies to a specific one. It is helpful in case if data with label is scant. Such approaches simultaneously mitigate the data demands and significantly shorten training time. Specifically, authors successfully classified mammographic tumors with transfer learning on only 607 full-field digital mammographic images, showcasing the capability of this approach to reach a high-performance when only little data is available [7]. Transfer learning has been successfully used for classifying images of macular degeneration and diabetic retinopathy in other studies [8]. But transfer learning adds challenges on creating adapted models. With fine-tuning being an essential step in this process, it is crucial to decide which layers of the pre-trained model will be re-trained and which will be frozen. This decision is crucial to ensure that the performance of the model is optimized for the desired task without obliterating information learned by the model during initial model training [9].

MT is considered one of the most promising breakthroughs to overcome language barriers but has been proven dependent on the deep learning model (DLM), the backbone of the MT system. Deep learning based (usually hosted in cloud) MT has latency issues and stresses the network. Although edge and end-based computing can provide low-latency, real-time translations, do not have the resources to utilize large models like GPT-3 or Yuan 1.0 with high efficiency [10]. Furthermore, continual adaptation and the need for privacy in sensitive applications like healthcare make cloud-based MT solutions even more challenging. These problems can be solved with Edge AI method, where data would be processed in proximity to its source, thus, reducing the time taken to communicate and hence improving responsiveness in translating [10]. For edge servers, the predominant literature around pioneering research is the physical resource obstacles that a huge and complex model like mBART (and all the other most fashionable NMT frameworks) imposes.

Similar to the challenges faced by MT systems, large language models (LLMs) also have deployment problems. This relates to both their energy demands which have environmental consequences and also the expanse they are vulnerable to adversarial attacks and then, their slow real-time responsiveness making them less useful for chatbots and translation services

[11]. Edge AI has a solution to that, it is an architecture model that talks about where the data should be processed, given that it allows to deliver data from the edge, for more secure data transfer and, at the same time, saving some energy by not having to transfer data. Three key phases of ML/DL model deployment, device cost, model performance, and data privacy, affect the deployment in companies developing software-intensive embedded systems according to a multi-case study [12] of seven companies. The most important cost is the device cost, centralized cloud architectures are cost-effective, while edge deployments are too costly to deploy, where resources have to be managed carefully along with the device environment, particularly if it is on battery. The low-cost edge devices were often not enough capable in terms of deployment. The impacts of model performance were also a contributing factor, as centralized architectures were able to exploit global optimization, while decentralized architectures allowed for better local performance through the leveraging of transfer learning approaches. A specific struggle mentioned was the balance between having global consistency vs. local optimization. Data privacy - especially in parts of the world with strict regulations, such as the EU - made it difficult to strike the right balance between obtaining what is needed to improve models and complying with privacy legislation, ultimately capping the potential of AI solutions in tightly regulated markets.

Another challenge of deployment of models on mobile and embedded devices is due to differing computing environments of mobile and embedded devices. Many state-of-the-art deep learning examples lack immediate portability to mobile and embedded devices due to differing hardware and software architectures than traditional computers. This difference requires to design specific models and optimization methods to be able to exploit mobile and embedded hardware capabilities [13]. In addition to that, high-end models can be expensive in terms of required hardware resources during inference, which makes leading a challenge for deployment, especially when it comes to edge, weak-scaled devices. In cases where processing is to be done at the same time, models need to be optimized to run without getting constrained by available hardware [14].

Existing literature has pointed out that there are several industrial challenges with personalization services based on recommender systems when deployed in real world. It is resource-intensive, large models need lots of computing power to train and inferencing as

well, so this is another challenge for easy adoption. When fine-tuning these models to improve personalization performance, such a demand is only further compounded, necessitating efficient strategies like LoRA [15] and QLoRA [16]. Online services also have requirements for fast response. To that end, solutions like pre-computing embeddings to store in vector databases are being investigated to balance performance and reduce latency. In addition, domain-specific data is also cumbersome to acquire: it is heterogeneous, messy, and noisy, rendering collection and filtering a laborious process in contrast to the acquisition of common-sense knowledge. As a result, these problems not only bring unnecessary training redundancies but raise cost of labor, and therefore worsen the deployment of functional personalization systems [17].

With Hindi spoken by 43.6% of the population of India alone, in addition to its prevalence in the rest of the world [18], advancing machine translation for this language pair is of utmost necessity for better communication globally and overcoming linguistic challenges. There are three main approaches for Hindi-English MT. SMT models use a statistical approach, generating translations by first aligning phrases or words statistically from bilingual corpora, while being heavily dependent on large datasets for translation, often ignoring linguistic nuance and confronting limited data availability for Hindi-English. NMT, based on models of Recurrent Neural Networks (RNN) and Long Short-Term Memory (LSTM) networks, creates translations with architecturally based encoder-decoder systems in which the network learns patterns of a sentence through its training. However, it struggles with long sentences and needs extensive training data, which limits its effectiveness for low-resource pairs like Hindi-English. Hybrid Machine Translation (HBMT) combines rule-based methods with statistical or neural models, leveraging both linguistic rules and data-driven approaches to improve translation accuracy, though it is computationally complex and difficult to implement [19]. Additionally, recent advancements highlight that Transformers outperform RNN and LSTM models by using self-attention mechanisms instead of recurrence. This allows Transformers to learn context more efficiently, process sequences in parallel (unlike RNNs which are sequential). Moreover, Transformers avoid issues like vanishing gradients, making them more effective for translation tasks, achieving better performance with faster and more efficient training,

especially when handling long and complex sentences [20]. This is the reason transformer encoder-decoder based model, mBART, was used for the research.

However, there are significant gaps in research in the application of PEFT in Hindi-to-English machine translation (MT) which will be addressed. A significant problem is that low resource languages such as Hindi are not as well represented in the literature as common language pairs such as English-French and English-German. While Hindi is among the most common languages in the world, it is low-resourced in terms of idioms. This limitation hinders the effective translation of such expressions without any dedicated datasets for idioms. Due to this limitation, PEFT techniques are not utilized to their full potential while translating such culturally specific phrases in Hindi-English MT. LoRA and QLoRA have enabled a significant reduction in fine-tuning overhead, however little attention for Hindi-English translation has been received, with most studies so far focusing on NLP tasks like text classification [21] and summarization [22]. This is particularly interesting in light of the morphological richness of Hindi—its verb inflections and compound-words—and the syntactic disparity between its SOV (Subject-Object-Verb) structure and that of English, which is SVO (Subject-Verb-Object). All of these issues make it difficult to obtain high quality translations using existing fine-tuning approaches and indicate that further research is warranted to develop adaptations for these techniques on this language pair. The synergistic effect of integrating multiple PEFT techniques is another major gap in Hindi-English MT. Although these techniques aim to be cost-effective in terms of computation, little research has been done on how these techniques can be fused without deteriorating the overall translation quality. While there are a number of multilingual pretrained models, e.g., mBART [23] and mT5 [24] have been used for the Hindi-English MT task, the exploration of applying PEFT techniques like distillation, LoRA, and QLoRA in particular for these models is limited. Data scarcity, linguistic differences between Hindi and English, create a need for more specialized approaches.

This work aims to narrow gaps by optimizing the balance between translation, memory allocation, time utilization during training and the inference time with the help of various PEFT methods for Hindi-to-English translation. This work could significantly contribute to the machine translation of various other language pairs. Additionally, it may provide valuable insights to researchers on the benefits of these recently developed PEFT

techniques in the context of machine translation. For investigating trade-offs between performance and resource efficiency, specifically memory usage, training time, and inference time, two PEFT techniques - LoRA and QLoRA - were employed. The transformer-based mBART model was fine-tuned using these techniques for Hindi-English translation. A varied Hindi-English parallel Corpus was constructed, consisting of sentence pairs spanning multiple linguistic dimensions, varying degrees of grammatical complexity, like simple vs. compound, or active vs. passive voice, and direct vs. indirect speech, as well as phrases with idiomatic use were included. Further, the dataset covered diverse contextual domains such as transliteration, news articles, technical content, and sentences across different tenses.

1.2 Literature Review

This section is about literature that connects PEFT with MT. AI-based MT systems are a rapidly evolving field of research. It has the aim to improve efficiency and accuracy, especially under constraints such as memory, latency, and training time. The application of PEFT to MT with resource languages like Hindi is not explored well. A few key works have started bridging this gap. Ou et al. [25] tuned the Llama2-13B model with NEFT and QLoRA for sentiment analysis of finance with a 0.8392 accuracy, performing better than the variants of RoBERTa and DeBERTa. Although non-MT, this showed the ability of QLoRA for domain-specific adaptation. Zhuohan Li et al. [26] suggested a "Train Large, Then Compress" approach based on large Transformer models in WMT14 English–French translation. They demonstrated that models that were briefly trained and subsequently heavily compressed not only accelerated convergence but also had better BLEU scores than smaller baseline models with 60–70% memory savings and 2–3x faster inference via 6-bit quantization [26]. He et al. [27] introduced a rapid and precise NMT model utilizing translation memory (TM) in a Transformer architecture. Their model employed a single bilingual sentence as TM, trained with a new training criterion. On translation tasks like WMT17 Zh→En, they obtained a BLEU score of 28.3 and cut training time per epoch by 75% (from 17,841s to 4,518s) while having only an 8.96% increase in parameters [27]. Inference time was improved 2–4x.

Liu et al. [28] countered MT training inefficiencies by employing a tanh-activated bi-directional RNN that allows for better contextualization of past and future context. For the

translation from German to English, they reported a 22.77 BLEU score with a 66% decrease in training time (8.5h vs. 23h), while for English to French, they reported 18.52 BLEU, improving upon OpenNMT by 3.13 BLEU points [28]. A few works have emphasized improving model performance within computational limits, which corresponds with PEFT's objective of minimized resource reliance. EdgeTran [29] merged device-aware co-search with surrogate models and BOSHCODE to minimize model size by 2.8x, latency by 15%, and energy consumption by 10x over BERT-Base. Park et al. [30] presented methods such as FlashAttention, speculative decoding, and continuous batching to maximize inference on AI accelerators. MELO [31] illustrated the potency of LoRA in medical image diagnosis, showing diagnostic performance that was equivalent to full models yet with only 0.17% trainable parameters and 0.5MB extra storage space per task. Likewise, Zhang et al. [32] introduced an end-edge-cloud structure for AI-generated content (AIGC) services based on federated learning and deep mutual learning for fine-tuning massive models on decentralized edge devices. Although originally targeted at medical use such as X-ray diagnosis, the method enhanced diagnostic performance but degraded with performance as more federation nodes were introduced, which points to the difficulty of scaling PEFT in distributed settings.

In various applications, PEFT methods have indicated potential in generalization, latency saving, and parameter reduction. MTLoRA [33] used LoRA in a multi-task learning framework and achieved the improvement of accuracy over full fine-tuning at the cost of decreasing trainable parameters by 3.6x. IISAN [34] proposed a decoupled PEFT framework for multimodal recommendation tasks, with an accuracy of 46.74% and a decrease in trainable parameters by 97.89%, as well as 59.59% decreased training time. Chakraborty et al. [35] applied PEFT to detect code vulnerabilities and observed drastic performance degradations (as much as 95 percentage points in precision) on real-world datasets, which were alleviated to some extent with data augmentation (30% improvement). Other pertinent works are Maatouk et al. [36], who examined LLM usage within the telecom industry, highlighting issues such as explainability and stability—important factors in using LLMs for real-time MT. Zhang et al. [37] suggested a skip-gram and edge integration method for MT that achieved accuracies of improvement at 10.79% and more than 10% improvement in other measurements such as matching time and

complexity. Bhardwaj et al. [38] presented an in-depth survey of LLM deployment on the edge, with emphases on energy and resource limitations, which are concordant with the objectives of memory-efficient MT. Yu et al. [39] surveyed visual tuning methods for transfer foundation models, with discussion on generalization trade-offs applicable to cross-domain PEFT tuning. Table 1 below summarizes the studied literature and their relevance to our work.

Table 1. Summary of Key Literature on PEFT and Machine Translation

Reference	Key Contribution	Model/Technique	Results / Insights	Relevance
[25] (Ou et al., 2024)	Finance domain tuning	QLoRA, NEFT	0.8392 accuracy, > DeBERTa	PEFT in domain adaptation
[26] (Li et al., 2020)	MT via compression	Large Transformer + pruning	BLEU: 35.0 (En→Fr), 50% faster training, 60–70% memory reduction, 2-3x faster inference	Direct PEFT use in MT
[27] (He et al., 2021)	TM-integrated NMT	TM + Transformer	BLEU: 28.3 (En→De), 68.49 (Es→En), +1.0 over baseline, 75% reduction in training time, and 2-4x faster inference from 1.91s to	Hybrid MT architecture with efficiency

			0.41s per sentence.	
[28] (Liu et al., 2020)	Bi-RNN NMT speed-up	tanh-activated Bi-LSTM	BLEU: 22.77 (De→En), 18.52 (En→Fr); 66% faster training	MT-specific training efficiency
[29] (Tuli et al., 2023)	Optimized Transformer search	BOSHCODE, GPTran	2.8x smaller models, 10x lower energy use	Edge-device efficiency for MT scaling
[30] (Park et al., 2024)	LLM inference optimization	FlashAttention, batching	Lower latency, optimized compute usage	Efficient inference tech for MT integration
[31] (Zhu et al., 2024)	LoRA for diagnosis	LoRA, ViT	0.17% params, 80–99% accuracy	Low-param PEFT under constraints
[32] (Zhang et al., 2024)	AIGC on edge for medical use	Federated + mutual learning	Improved diagnosis, latency; but degradation with more nodes	Distributed model fine-tuning relevance
[33] (Agiza et al., 2024)	LoRA for MTL	MTLoRA	3.6x parameter reduction, accuracy boost	Demonstrates PEFT multi-task capability
[34] (Fu et al., 2024)	Multimodal PEFT	IISAN	46.74% accuracy, - 97.89% params, -	Efficient PEFT model for complex tasks

			59.59% training time	
[35] (Chakraborty et al., 2024)	Vulnerability detection via PEFT	PEFT, augmentation	Up to 95pp drop in precision, 30% recovery via augmentation	Generalization challenges, PEFT tuning insights
[36] (Maatouk et al., 2024)	LLM use in telecom	Semantic similarity, automation	Identified key challenges (hallucination, explainability)	Cross-domain LLM deployment relevance
[37] (Li et al., 2024)	Edge-based MT system	Skip-gram, edge integration	+10.79% accuracy, +10.78% matching factor, -12.16% error	MT-oriented, system-level improvements
[38] (Bhardwaj et al., 2024)	Survey on LLMs at edge	LLM optimization survey	Latency, privacy, scalability issues	Aligns with inference constraints in MT
[39] (Yu et al., 2024)	Visual foundation model tuning	Prompt/adapter tuning	Efficiency vs. generalization tradeoffs	Cross-domain PEFT insights

Although prior studies have achieved much progress in machine translation based on AI and in allied fields, some gaps exist, especially with respect to harnessing state-of-the-art fine-tuning and optimization methods. For instance, while parameter-efficient techniques such as LoRA and QLoRA have proven useful in alleviating training overhead and enhancing model deployment in applications such as finance, medical imaging, and recommendation systems, their direct application to neural MT tasks—particularly those

involving multilingual models such as mBART for Hindi-English translation—remains underinvestigated. In addition, models such as MTLORA and IISAN, while achieving success in multi-task and multimodal environments respectively, have not been benchmarked against each other or studied in the context of MT for tasks involving idiomatic, domain-specialized, or tense-varying translations. There is also limited research testing how inter-changeable or tunable LoRA and QLoRA are in low-resource MT environments. In addition, while a few optimization techniques such as quantization and edge-aware pruning have been put forward, paged optimization or fine-grained memory compression remains largely unexplored during PEFT operations in MT.

Based on such voids, this study examines the parameter-efficient fine-tuning of pre-trained mBART models with a focus on LoRA and QLoRA. Although numerous PEFT algorithms have been compared in applications such as finance and recommendation systems, less is known about direct comparisons between these algorithms within machine translation. This research meets that gap by performing controlled comparative experiments between LoRA and QLoRA, investigating hyper-parameter tuning and the trade-offs among memory efficiency, training time, inference time, and translation quality. Empirical results provide rich information that supports the construction of scalable and efficient multilingual machine translation systems for both academic and industrial applications. Table 1 presents the comparative overview of all the pieces of work dealt with in this literature review based on their contribution, methods utilized, and results.

CHAPTER-2

METHODOLOGY

This study takes a systematic approach to enhance existing translation frameworks demonstrated in Figure 1. It begins with a diverse dataset collection and preprocessing, such as data splitting and tokenization. The mBART base model is taken as a foundation model and is fine-tuned with computational and storage optimization. Hyperparameter selection was also part of fine-tuning the model for domain-specific translation tasks. And resulting mBART Fine-Tuned Model was evaluated on quantitative metrics — METEOR and ChrF, and some deployment considerations — time to train, response speed and storage needs. A Pareto-optimal solution is attained through this methodology that strives to balance the quality of the translation with practical constraints related to deployment.

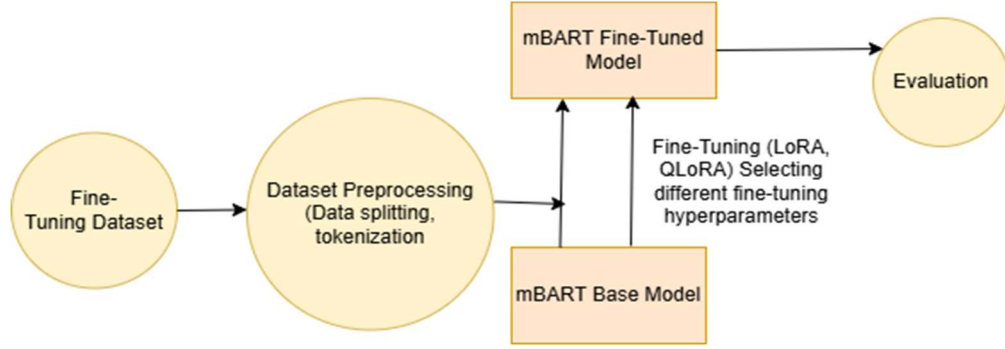


Fig. 1 Research methodology for refining translation model

A sequence of the most suited parameters for fine-tuning a Hindi-English translation model is proposed here. The flow of the experimental procedure consists of four stages: data collection, data preprocessing, model preparation, and model evaluation. A Hindi-English bilingual dataset was created that covers all possible linguistic structures and contextual nuances. Dataset preprocessing was followed by splitting then tokenization. The mBART base model was fitted using common parameters using powerful techniques- LoRA and QLoRA to benefit. Last but not the least, the metrics used for evaluation of the fitted model were ChrF, METEOR and training/inference time, etc.

2.1 Data Collection

The dataset is well-structured for Hindi-English translation research and is further subdivided into three major categories which are: Standard Sentences, Idiomatic Expressions, and Voice and Speech Constructions, with a total of 9,900 sentence pairs. It has multiple sub-categories like sentence type, tense, and so on for each primary category. The Standard Sentences and Idiomatic Expressions into Declarative, Exclamatory, Imperative, Interrogative, Negation, Complex, and Compound sentences, and each sentence type are divided into nine tenses, where there are 50 sentence pairs per tense, so 3,150 pairs for each category. Voice and Speech Constructions include Active Voice, Passive Voice, Direct Speech, and Indirect Speech, each also divided into nine tenses, with 100 sentence pairs per tense (50 technical and 50 news context), totaling 3,600 pairs. The translations of Hindi to English were self-prepared by a team of 3 fluent in both Hindi and English. If in doubt, translations were discussed to ensure consistency and correctness. Although double efforts were made to ensure the accuracy of the translations, no external expert validation was undertaken. It might be a limitation of the dataset. Team members reviewed and re-reviewed translations several times to ensure translation quality. Ambiguous or difficult sentences were addressed and resolved collectively. Various news article sources, technical article sources, and idioms websites were referred. This diverse and comprehensive dataset supports various grammatical structures, tense forms, and speech types, enhancing machine translation models' accuracy and contextual understanding.

The dataset also contains transliterations. Transliteration is the process of taking words written in one script and writing them using the letters of another script so the pronunciation stays similar. Here are some examples of transliterations from the dataset:

वे सिस्टम को कॉन्फ़िगर कर रहे होंगे।, They will be configuring the system.

हम सुरक्षा प्रोटोकॉल का परीक्षण कर रहे होंगे।, We will be testing the security protocols.

वह एप्लिकेशन को डिबग कर रहा होगा।, He will be debugging the application.

वे वेबसाइट का अनुकूलन कर रहे होंगे।, They will be optimizing the website.

The distribution of sentences by primary categories and tenses is given in Table 2 below, by standard sentences and idiomatic expressions in Table 3 below, and by Voice and Speech Constructions in Table 4 below.

Table 2 Sentence Distribution by Primary Categories and Tenses

Primary Categories	Sub-Categories	Tenses	Sentence Pairs per Tense	Total Sentence Pairs
Standard Sentences	7 Sentence Types	9	50	3,150
Idiomatic Expressions	7 Sentence Types	9	50	3,150
Voice and Speech	4 Voice/Speech Types (2 contexts)	9	100	3,600
Total	-	-	-	9,900

Table 3 Standard Sentences and Idiomatic Expressions Breakdown

Sentence Type	Sentence Pairs (per tense)	Total Sentence Pairs (per type)
Declarative	50	450
Exclamatory	50	450
Imperative	50	450
Interrogative	50	450
Negation	50	450
Complex	50	450
Compound	50	450
Total (per category)	-	3,150

Table 4 Voice and Speech Constructions Breakdown

Voice/Speech Type	Sentence Pairs (per tense, per context)	Total Sentence Pairs (per type)
Active Voice	100 (50 technical, 50 news)	900
Passive Voice	100 (50 technical, 50 news)	900
Direct Speech	100 (50 technical, 50 news)	900
Indirect Speech	100 (50 technical, 50 news)	900
Total	-	3,600

2.2 Data Pre-processing

The dataset underwent essential preprocessing steps to ensure effective training of the machine learning models and accurate translation results. Initially, the dataset was split into training, validation, and test sets in a 70:15:15 ratio, maintaining a balanced representation of each category and sub-category to ensure the model is trained, validated, and then tested on the data split comprising of each linguistic variation the dataset consisted. The training data split and validation data split were used while adapting the base model, the model parameters were updated according to its performance on the train data split, and after every epoch, the model with changed parameters was tested for its performance on validation data split. In the end, when all epochs were completed, the model was examined on the testing data split, which the model had never seen before.

Since the mBART model, like any other model, is a mathematical model. So, it requires the input that is in string format to be converted to numbers with which computation can be performed. Also, the output generated by the model is also number. To perform the conversion of input string to numbers or output numbers to translated text a tokenizer is required. Tokenization was performed using the MBart50 Tokenizer. This method is advantageous for languages with rich morphology, such as Hindi, as it can manage unknown words by breaking them into known subword units. Table 5 below mentions the standard preprocessing parameters applied to the Hindi-English parallel corpus. These parameters, including tokenization method, sequence length limits, and padding strategy, were kept consistent across all model variants to ensure fair comparison between different fine-tuning approaches.

Table 5 Common data preprocessing parameters applied to the Hindi-English dataset across all experimental configurations

Parameter	Value of Parameter
Input tokenization	MBart tokenizer
Maximum length	128 tokens
Padding strategy	max_length
Truncation	True
Dataset splits	train, validation, test

2.3 Model Preparation

Following the preprocessing stage, the mbart-large-50-many-to-many-mmt existing model available on Hugging Face has been utilized as the foundational model in this research. This model is a fine-tuned checkpoint of mBART-large-50 [40]. It is based on BART [41], a denoising autoencoder model for language generation tasks. Table 6 presents the standard training hyperparameters used across all model configurations. These base parameters were kept constant while LoRA and QLoRA-specific parameters were varied for experimental comparisons.

Table 6 Common training hyperparameters for Hindi-English mBART translation model

Parameter	Value of Parameter
Base Model	facebook/mbart-large-50-many-to-many-mmt
Learning rate	1e-4
Train batch size	8
Eval batch size	16
Warmup steps	500
Weight decay	0.01
Maximum sequence length	128 tokens
Evaluation strategy	Per epoch
Save strategy	Per epoch

Number of Epochs	5
------------------	---

The two key techniques experimented are described below:-

LoRA- Using LoRA, low-rank adapters were trained, allowing to fine-tune the model without modifying the original parameters extensively [15].

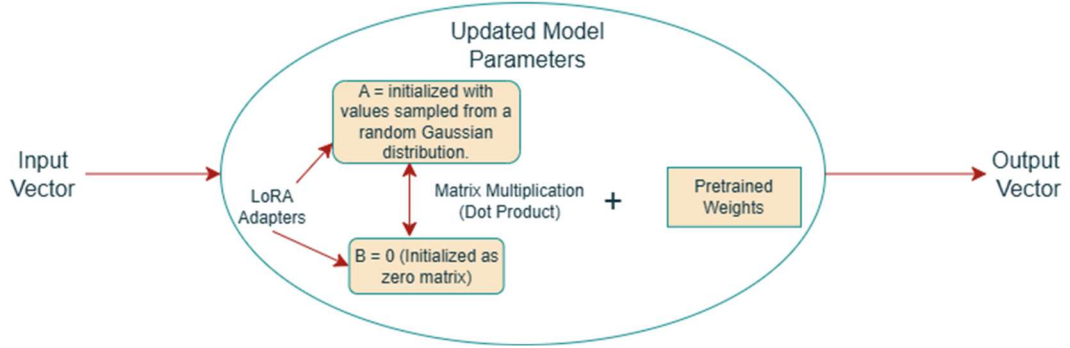


Fig. 2 Reparameterization performed in LoRA technique. [15]

LoRA possesses several key advantages. The common part of the model can be frozen, and to change the task the model is required for, matrices A and B in Figure 2, need to be replaced. This reduces the memory needs and task-switching overhead. Training is done faster since computation of the gradients for most parameters is not needed. It also introduces no additional inference latency compared to a fully fine-tuned model and also reduces the memory needs of the model when fine-tuning because the optimizer states and the checkpoints for frozen layers need not be stored [15].

QLoRA- This technique initially quantizes the base model to reduce memory requirements and since the quantized model would lack in accuracy, fine-tuning on task-specific data would be required for which LoRA fine-tuning technique is applied, see Figure 3. QLoRA introduces multiple innovations designed to reduce memory use without sacrificing performance: (1) 4-bit NormalFloat, an optimal quantization data type for normally distributed data that yields better empirical results than 4-bit Integers and 4-bit Floats, (2) Double Quantization, a method that quantizes the quantization constants, (3) Paged Optimizers, to avoid the in-memory surges that occur when processing a large amount of data [16].

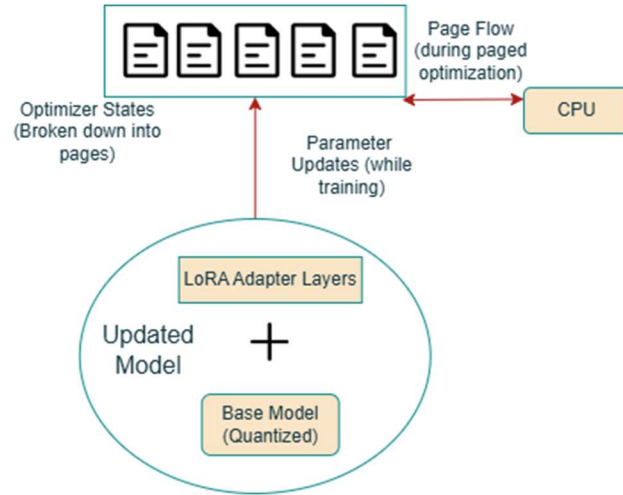


Fig. 3 QLoRA compresses the model, refines the model using LoRA and uses the concept of paging to handle memory spikes, providing better memory efficiency than LoRA. [16]

2.4 Evaluation Metrics Used

For evaluating Hindi-to-English translations, a combination of the METEOR and ChrF metrics were employed, selected for their advanced capabilities in capturing linguistic nuances and morphological variation, which are particularly relevant for this language pair. Table 7 lists the evaluation metrics and computational resources tracked during model training and inference. These measurements enable comparison of model performance and efficiency across different configurations. The average training time per epoch was calculated by measuring the duration between epoch start and end using a custom timer callback. The average inference time was then calculated by measuring the translation time for each sentence in the test set.

Table 7 Performance evaluation metrics and computational resource measurements tracked across model variants

Parameter	Value of Parameter
Primary metrics	METEOR, ChrF
Performance tracking	Loss
Model selection	Best model based on loss

Original model GPU memory	MB
Fine-tuned model additional GPU memory	MB
Average training time per epoch	seconds
Average inference time	seconds
Trainable parameters	Count

METEOR tests translations by matching machine-generated text with reference translations using multiple techniques: exact matches, stemming, synonyms, and paraphrases. It calculates precision, recall, and an F-score with an explicit word-order penalty, providing a nuanced assessment of semantic alignment while accommodating the differences in word forms and syntactic structures common in Hindi and English.

ChrF (Character F-score), in contrast, assesses character-level overlap, which is advantageous for high-morphology languages like Hindi. Unlike METEOR, which primarily relies on word-level matches and stems, chrF captures fine-grained morphological differences, such as tense and aspect variations (e.g., dancing vs. danced), enhancing the evaluation of word-form nuances. METEOR score can make this mistake of assuming dancing and danced to be the same since it performs stemming and converts both to their same root form i.e. dance. Since chrF score does not perform stemming it would not make this mistake.

Other commonly used metrics such as BLEU and ROUGE-L are widely recognized, but are limited to basic n-gram matching and thus lack the sophistication required for a comprehensive evaluation of translation quality. BLEU, for example, relies solely on n-gram precision without accounting for synonymy or flexible word ordering, which can lead to inaccurate assessments of semantic quality, especially in translations between syntactically diverse languages like Hindi and English. By nature in similar fashion, ROUGE-L is based on longest common subsequence (LCS), which again does not indicate much knowledge of semantic or morphological correctness.

2.5 Platforms and Frameworks Used

Experiments utilized the Google Colab cloud platform employing NVIDIA T4 with CUDA support, Python code was written in Jupyter Notebook format. The base framework used the facebook/mbart-large-50-many-to-many-mmt model via Hugging Face's transformers library. Data processing was implemented using pandas library for CSV operations and datasets library was used for seamless handling of data. The torch library was used to run GPU computations in the training pipeline and peft library to implement parameter-efficient fine-tuning techniques. It was required to quantize the model to implement QLoRA, which was done through the BitsAndBytesConfig library. Lastly for the evaluation metrics, we computed METEOR scores using nltk library and chrF scores using sacrebleu library, measuring the translation quality between Hindi and English. The results and model were saved in google drive using google. tqdm library progress display integrated with colab framework.

CHAPTER-3

RESULTS

This section provides an overview of the results, showing that the optimizations achieve high-quality translations with reduced computational costs. These results ideally improve ease of use and common use of Hindi-English translation systems, opening the door for translation in lightweight models for end system usage, like a mobile, live translation in a real-time usage like broadcast and video conferencing, easy availability in instant messaging platforms, or batch translations for enterprises. Notably, memory use, training time, and inference time become important for real-world use. The decreased memory footprint enables the translation model to be run on low-end hardware, which makes deployment possible on edge devices or lower-cost servers—this is a significant consideration for mainstream usage in areas with limited access to more expansive hardware. Shorter training times are essential for speeding up experimentation with different model architectures, refreshing models with new patterns of language, and adjusting to changes in language in the real world. Most notably, a reduced inference time translates into better user experience with instant translations. Fast inference ensures seamless natural conversations with high expectations of modern users for everything from real-time conversations to document translations and website localization.

Table 8 Comparative Performance Analysis of Fine-Tuning Approaches: Full, LoRA, and QLoRA

Models / Paramet ers	Model GPU memory (in MB, Mega Bytes)	Average training time per epoch (in seconds)	Average inference time (in seconds)	METEOR score	ChrF score
Full Fine Tuned Model	7008.164	887.55	0.4045	0.8239	0.7621

Lora Fine Tuned	2374.524	266.15	0.3715	0.8296	0.7707
QLora Fine Tuned	696.82	343.70	0.4037	0.8234	0.7652

The systematic investigation, conducted extensive experiments to optimize the LoRA and QLoRA training configurations. Evaluated different rank, alpha, dropout, and target module values to identify the optimal balance between training time, deployment concerns such as storage memory required, inference time, and performance of the model on Hindi-English language translation. Throughout this optimization process, each decision was informed by empirical results and careful consideration of trade-offs between performance and resource utilization. This section, discusses the relationships between the parameters and gives a pretty decent analysis of how these experimental hyperparameter configurations are impacting the model performance.

Table 8 compares the fine-tuning methods (Full, LoRA, and QLoRA), in terms of GPU memory usage; average training time per epoch; average inference time; and METEOR and ChrF scores on translation. It demonstrates the resource-economical and effective nature of LoRA and QLoRA compared to full fine-tuning. Figure 4 provides comparison between full fine-tuned, LoRA fine-tuned and QLoRA fine-tuned on the model GPU memory and average training time per epoch (lower is better); Figure 5 provides a comparison between full fine-tuned, LoRA fine-tuned and QLoRA fine-tuned on average inference time, METEOR score, and ChrF score (a higher METEOR score and ChrF score indicates a better result). It gives an idea of how each approach trades off performance and inference time.

CHAPTER-4

DISCUSSION

The systematic investigation, conducted extensive experiments to optimize the LoRA and QLoRA training configurations. Evaluated different rank, alpha, dropout, and target module values to identify the optimal balance between training time, deployment concerns such as storage memory required, inference time, and performance of the model on Hindi-English language translation. Throughout this optimization process, each decision was informed by empirical results and careful consideration of trade-offs between performance and resource utilization. This section, discusses the relationships between the parameters and gives a pretty decent analysis of how these experimental hyperparameter configurations are impacting the model performance.

4.1 Comparing performance of LoRA with full fine-tuning

LoRA (Low-Rank Adaptation), with parameters ($r=16$, $\alpha=512$, $\text{dropout}=0.0$, $\text{target_module}=[k_proj, v_proj]$), achieves a 66.12% reduction in GPU memory usage and 70.01% depletion in time to train in contrast with the fully fine-tuned model. It also shows marginal improvements in performance metrics (METEOR: +0.69%, ChrF: +1.13%). The inference time decreased by 8.16%, demonstrating LoRA's effectiveness in balancing computational efficiency with model performance.

4.2 Comparing performance of QLoRA with full fine-tuning

QLoRA parameters ($\text{bits} = 4$, $\text{double_quant} = \text{False}$, $\text{compute_dtype} = \text{bfloat16}$, $\text{paged_optimization} = \text{True}$), demonstrates a 90.06% reduction in GPU memory usage and 61.27% reduction in training time, its performance metrics remain remarkably close to the full fine-tuned model's performance metrics (METEOR: -0.06%, ChrF: +0.41%). Compared to the full fine-tuned model, QLoRA demonstrated a marginal 0.2% faster inference time. The difference in METEOR and ChrF scores are negligible and are just because transformer models are not deterministic, may give slightly different response for the same input, hence are not required to be explained.

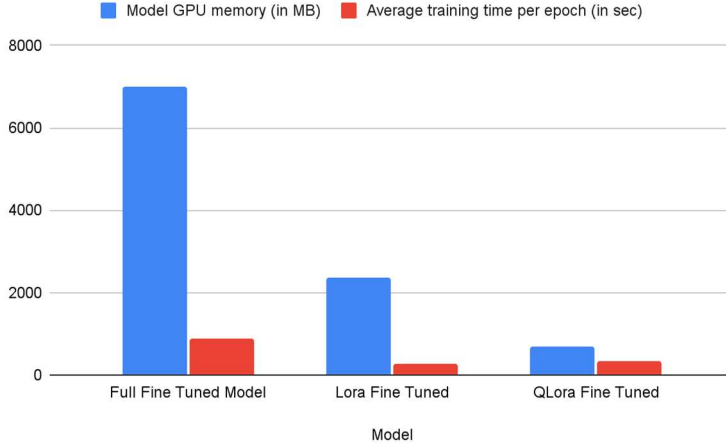


Fig. 4 GPU Memory and Training Time Comparison Across Fine-Tuning Methods

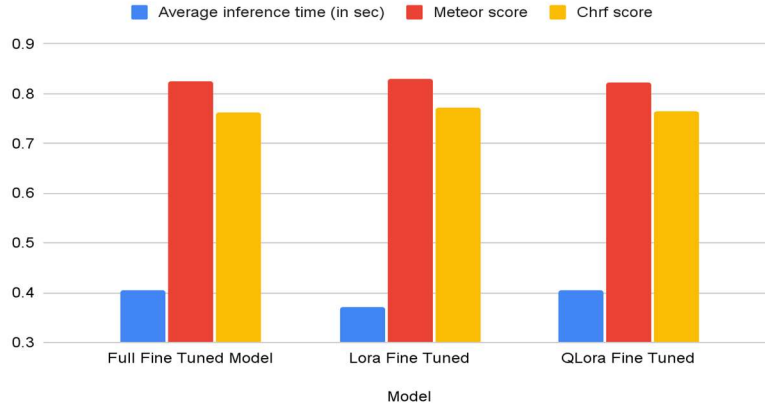


Fig. 5 Inference Speed and Translation Quality Scores (METEOR and ChrF) Across Fine-Tuning Approaches

4.3 Base Model Performance and why it needs Fine-tuning

The base model was not performing well in Hindi-English translation initially, as shown in Table 9, because at the core it lacked the understanding in its pre-trained data. Due to limited inclusion of Hindi specific linguistic patterns and domain specific words in the training process, it did not perform well on Hindi syntactic structure, morphology, idioms, tenses, and contextual knowledge for the domain, in Hindi content such as news articles, technical documentation and idioms. In order to tackle these limitations, the first step was to apply full fine-tuning, which brought a huge gain in translation quality, with METEOR and ChrF scores reaching 0.8239 and 0.7621 respectively, as seen in Figure 6. However,

this improvement came at a substantial computational cost, requiring approximately 4676.89 MB of additional GPU memory (see Figure 7) and an average training time of 887.55 seconds per epoch, as mentioned in Table 10. Table 11 and Table 12 show how LoRA and QLoRA improved the translations.

Table 9 Translation response improvement seen from base model to fully fine-tuned model

Source Text	Reference Translation	Base Model Output	Fully Fine-Tuned Output	Observation
हम भोजन कर रहे होंगे!	We will be having dinner!	We shall eat!	We will be eating food!	Correct tense captured by fine-tuned model.
क्या तुमने खाना खा लिया था?	Had you eaten the food?	Did you eat?	Had you eaten the food?	The output extracted by fine-tuning matches tenses perfectly.
वह उल्टा चोर कोतवाल को डाँट रहा था!	He was blaming others for his own faults!	On the contrary, the thief was beating the Kotwal.	He was accusing others of his crime!	The output by the fine-tuned model is aligned by context.
उसे ऊँची दुकान फीका पकवान मिला!	He got all show and no substance!	He found a flour cooker in the high store.	He found all that glitters is gold!	Fine-tuned output gets better, but misinterprets the idiom.

क्या वैज्ञानिकों ने नई खोज की है?	Have the scientists made a new discovery?	What have scientists discovered?	Has scientists made a new discovery?	This fine-tuned output has improvements but still contains a grammatical error! (Has vs. Have)
टीम नई सुविधाओं का परीक्षण करती है।	The team tests the new features.	Tests new facilities.	The team tests new features.	The output of fine-tuning shows fixed grammar and context.

Table 10 Comparative Power and Resource Utilization of the Base Model and Full Fine-Tuned Model

Models / Parameters	Model GPU memory (in MB, Megabytes)	Average training time per epoch (in seconds)	Average inference time (in seconds)	METEOR score	ChrF score
Base Model	2331.274	-	0.314512	0.645622	0.583080
Full Fine Tuned Model	7008.164	887.557712	0.404523	0.823913	0.762182

Table 11 Comparison of translation response of base model and LoRA fine-tuned model

Source Text	Reference Translation	Base Model Output	LoRA Fine-tuned Model Output	Analysis
-------------	-----------------------	-------------------	------------------------------	----------

हम भोजन कर रहे होंगे!	We will be having dinner!	We shall eat!	We will be having lunch!	Fine-tuned model captured the progressive tense.
क्या तुमने खाना खा लिया था?	Had you eaten the food?	Did you eat?	Had you eaten the food?	Fine-tuned model corrected the base model's simple past error by using the correct past perfect tense.
वह उल्टा चोर कोतवाल को डाँट रहा था!	He was blaming others for his own faults!	On the contrary, the thief was beating the Kotwal.	He was blaming others!	The metaphorical essence was lost; fine-tuned output simplified the meaning too much.
उसे ऊँची दुकान फीका पकवान मिला!	He got all show and no substance!	He found a flour cooker in the high store.	He found the high-profile shop delicious!	Both outputs were literal; fine-tuned version contradicted the intent by implying enjoyment instead of disappointment.
क्या वैज्ञानिकों ने नई खोज की है?	Have the scientists made a new discovery?	What have scientists discovered?	Have scientists made a new discovery?	Fine-tuned model matched the reference and fixed the base's interrogative mismatch.
टीम नई सुविधाओं का परीक्षण करती है।	The team tests the new features.	Tests new facilities.	The team tests new features.	Fine-tuned model accurately captured the subject-verb structure and corrected the translation of "features."

Table 12 Comparison of translation response of base model and QLoRA fine-tuned model

Source Text	Reference Translation	Base Model Output	QLoRA Fine-tuned Model Output	Analysis
हम भोजन कर रहे होंगे!	We will be having dinner!	We shall eat!	We will be eating!	QLoRA model correctly captured the progressive tense.
क्या तुमने खाना खा लिया था?	Had you eaten the food?	Did you eat?	Had you eaten the food?	Accurate translation with correct tense and full sentence structure.
वह उल्टा चोर कोतवाल को डाँट रहा था!	He was blaming others for his own faults!	On the contrary, the thief was beating the Kotwal.	The thief was scoffing at the opportunity!	QLoRA output misinterpreted the metaphorical phrase.
उसे ऊँची दुकान फीका पकवान मिला!	He got all show and no substance!	He found a flour cooker in the high store.	He got a flattened dish from the high store!	Incorrectly translated the idiom, missing its intended meaning of disappointment.
क्या वैज्ञानिकों ने नई खोज की है?	Have the scientists made a new discovery?	What have scientists discovered?	What have scientists discovered newly?	Incorrect word order; fails to maintain the meaning of the question.

टीम नई सुविधाओं का परीक्षण करती है।	The team tests the new features.	Tests new facilities.	The team tests new features.	Accurate translation with complete subject-verb structure and correct noun usage.
-------------------------------------	----------------------------------	-----------------------	------------------------------	---

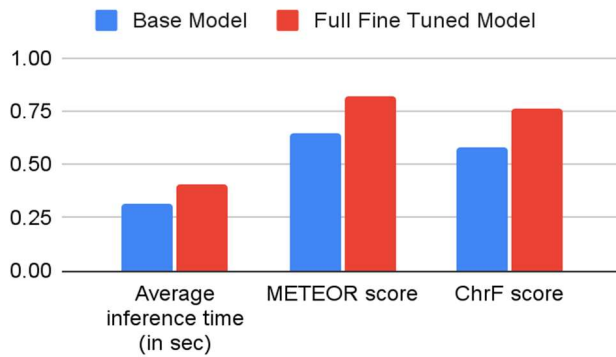


Fig. 6 Performance and Inference Time Comparison between Base Model and Full Fine-Tuned Model

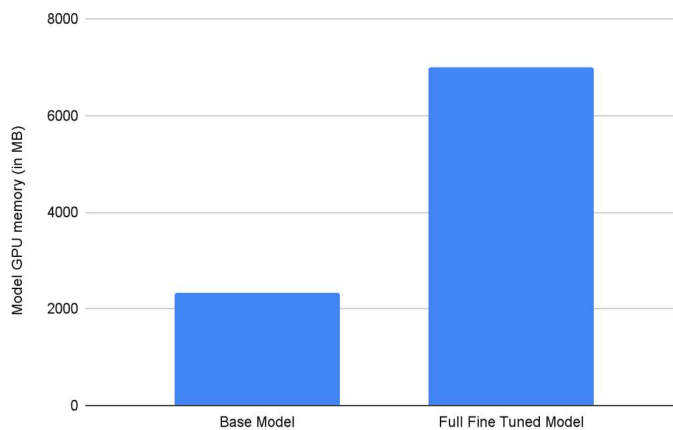


Fig. 7 GPU Memory Usage Comparison between Base Model and Full Fine-Tuned Model

4.4 Optimization Strategies for LoRA

Our optimization journey began with exploring different LoRA rank values. In Table 13 examining rank optimization, maintained consistent baseline parameters ($\alpha = 32$, $\text{dropout} = 0.1$, and $\text{target module} = [\text{q_proj}, \text{k_proj}, \text{v_proj}, \text{o_proj}]$) while testing ranks from 2 to 64. While rank value of 16 required 1021% more GPU memory (56.75 MB) than rank 2 (5.06 MB), it achieved a 0.5% improvement in METEOR score (0.802 vs 0.798) and a 0.55% increase in ChrF score (0.736 vs 0.732). Also, the inference time for rank 16 was relatively higher compared to other rank values, as shown in Figure 8, but, considering the significant gains in METEOR and ChrF scores, as well as the balanced memory and parameter trade-offs, rank 16 emerged as the clear winner. Higher ranks like 48 and 64 demonstrated diminishing or worse performance despite consuming significantly more memory (192.43 MB for rank 64, a 239% increase, as shown in Figure 9), parameters (14,155,776 parameters, a 300% increase), and training time per epoch (305.629 seconds for rank 48, a 4.83% increase compared to rank 16). Increasing the rank r leads to larger matrices A and B , thus directly increasing the memory requirement. This happens because the memory consumption of these matrices scales with r , and as r becomes larger, the memory overhead associated with storing these matrices and performing computations with them grows as well.

Table 13 Impact of LoRA Rank on Model Performance, Resource Utilization, and Trainable Parameters

LoRA Rank (r)	Additional GPU Memory (MB, Megabytes)	Avg. Inference Time (seconds)	METEOR Score	ChrF Score	Trainable Parameters (After LoRA)	Avg. Training time (seconds)
2	5.06	0.423	0.798	0.732	442,368	290.381
12	30.38	0.424	0.796	0.733	2,654,208	290.063
16	56.75	0.457	0.802	0.736	3,538,944	291.543
48	137.75	0.416	0.799	0.734	10,616,832	305.629
64	192.43	0.446	0.796	0.731	14,155,776	298.650

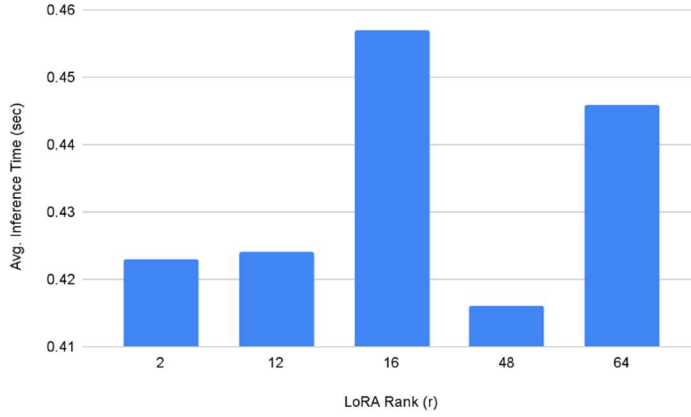


Fig. 8 Inference Time Comparison Across LoRA Ranks

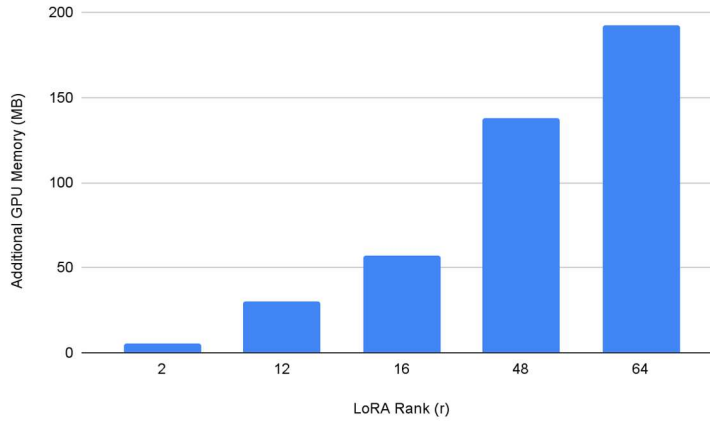


Fig. 9 GPU Memory Usage Across LoRA Ranks

Building on this foundation, the work moved to alpha optimization while maintaining a newly established rank of 16. Testing alpha values from 8 to 512, finding that an alpha value of 512 delivered the best results as shown in Figure 10, achieving a 4.4% improvement in METEOR score (0.820280 vs 0.785965) and a 5.3% increase in ChrF score (0.758328 vs 0.720382). Increasing alpha boosts the influence of the low-rank adaptations, allowing the model to learn more complex adjustments and achieve better task-specific performance. Remarkably, this substantial improvement in performance came without any increase in GPU memory usage, which remained steady at 56.751 MB across all alpha values, as evident in Table 14. The training time increased by only 3.07%

(297.398s vs 288.535s) when compared with the least training time found. Table 15 mentions some translation examples noticing which the improvement can be understood better.

Table 14 Impact of Alpha Values on Model Performance and Resource Utilization

Alpha	Additional GPU Memory (MB, Megabytes)	Avg. Training Time per Epoch (seconds)	Avg. Inference Time (seconds)	METEOR Score	ChrF Score
8	56.751	294.406	0.417	0.785965	0.720382
32	56.751	288.641	0.416	0.799623	0.735650
64	56.751	288.535	0.407	0.803014	0.740331
128	56.751	299.841	0.421	0.810165	0.748441
512	56.751	297.398	0.434	0.820280	0.758328

Table 15 Translation response improvement seen across models fine-tuned using different alpha values

Source Text	Reference Translation	Alpha = 8 Output	Alpha = 32 Output	Alpha = 64 Output	Alpha = 128 Output	Alpha = 512 Output	Observation
क्या अद्भुत सवारी होगी!	What a wonderful ride it will be!	What a wonderful ride!	What a wonderful ride it will be!	What a wonderful ride it will be!	What a wonderful ride it will be!	What a wonderful ride it will be!	Tense being corrected progressively; From Alpha = 32 onward outputs are correct.

जिम जाओ।	Go to the gym.	Go live.	Go to the gym.	Go to the gym.	Go to gym.	Go to the gym.	Alpha =32 and beyond (issues solved), Alpha = 128 (minor grammar polish required)
वह उम्र भर का साथ निभा रहा था।	He was keeping a lifelong companionship.	He had been playing along for years.	He was enjoying lifelong companionship.	He was enjoying lifelong friendship.	He was playing along for life.	He was serving a lifelong friendship.	Slight, incremental context enhancements, but misses the commitment in the input Hindi text which the last output fulfills.
उसने काम ऐसा किया कि साँप भी मर जाए और लाठी भी न टूटे।	He will have accomplished the task without any harm.	He worked in such a way that the snake died and the staff did not break.	He worked so hard that neither the snake died nor the staff broke.	He worked in such a way that neither the snake died nor the rod	He worked so hard that the snake died and the staff didn't break.	He worked in such a way that even the snake died and the staff	The idiomatic meaning has not been captured at all, though all alpha values, with literal translations being the norm. Refining it requires training for more time or epochs.

				broke .		broke off.	
वह कह रहा है कि राष्ट्रपति चुनाव में कड़ा मुकाबला हो रहा है।	He is saying that the presidential election is tightly contested.	He says that the Presidential is facing a tough challenge in the election.	He is saying that the president faces a tough challenge in the election.	He is saying that the presidential faces tough challenges in the election.	He is saying that the presidential election is facing tough challenges.	He is saying that there is a fierce struggle in the presidential election.	Progressive improvement in contextual accuracy; Alpha = 512 capture the sentiment accurately.
मुद्दों पर बहस की जा रही होगी।	Debates on issues will be being conducted.	Issues will be discussed.	Issues will be under discussion.	Issues will be under discussion.	The issues will be being discussed.	The issue will be being discussed.	Increase in grammatical accuracy; alpha = 128 and 512 preserve the progressive passive structure correctly.

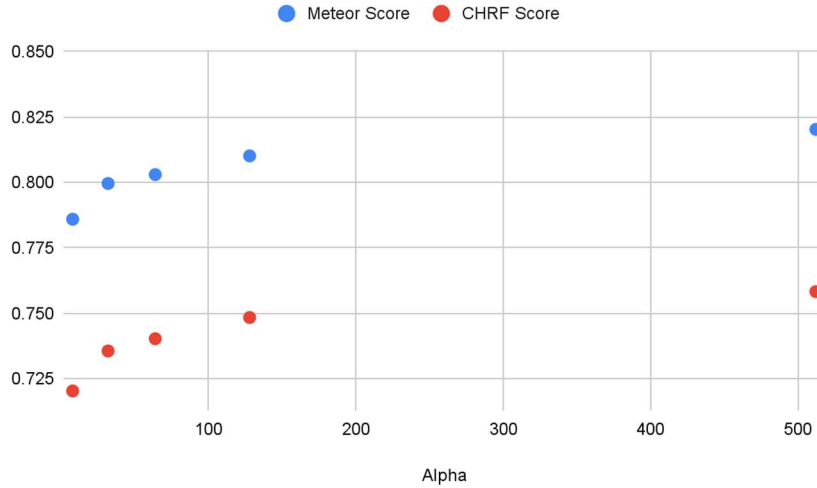
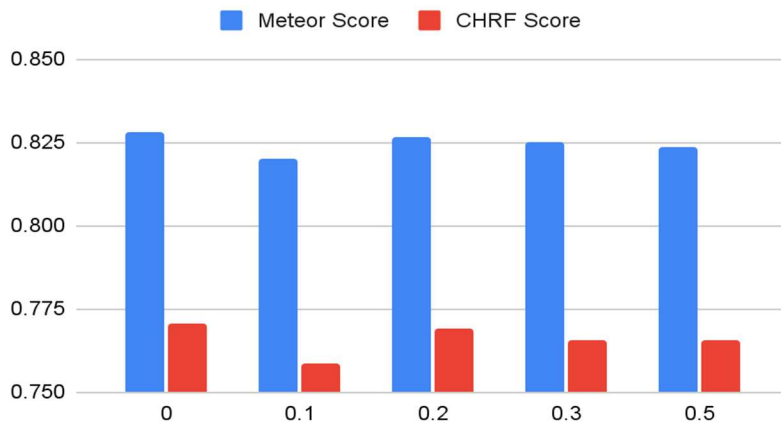
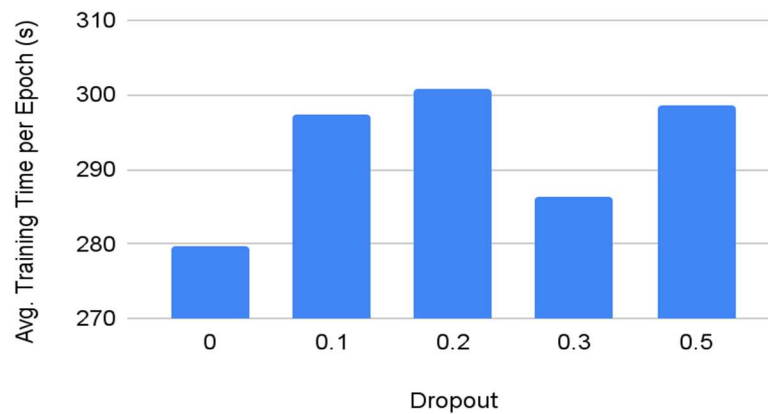


Fig. 10 Impact of Alpha Values on Translation Quality: Relationship Between LoRA Alpha Parameter and METEOR/ChrF Scores in Hindi-English Translation

Our next investigation into dropout rates brought perhaps the most surprising finding of this study. Dropout randomly disables neurons during training to prevent overfitting. Conventional studies suggests that some dropout is necessary to prevent overfitting, but the results told a different story. Testing dropout values from 0.0 to 0.5 while maintaining the optimized rank and alpha values, finding that a dropout of 0.0 actually delivered the finest performance on test data with respect to all metrics, as shown in Table 16. This configuration achieved a 0.92% improvement in METEOR score (0.827864 vs 0.820280) and a 1.63% increase in ChrF score (0.770722 vs 0.758328), see Figure 11, while reducing training time by 7.52% (279.666s vs 300.711s), see Figure 12. The reason of these results is that using no dropout allows the model to use all of its parameters for learning, leading to richer representations, better task adaptation and potentially faster convergence. Also, no dropout allows the model to process inputs during inference more quickly without the extra computational overhead that dropout introduces during training.

Table 16 Effect of Dropout on Training and Evaluation Metrics

Dropout	Avg. Training Time per Epoch (seconds)	Avg. Inference Time (seconds)	METEOR Score	ChrF Score
0.0	279.666	0.407	0.827864	0.770722
0.1	297.398	0.434	0.820280	0.758328
0.2	300.711	0.421	0.826811	0.768997
0.3	286.276	0.411	0.825085	0.765697
0.5	298.679	0.417	0.823504	0.765586

**Fig. 11** Evaluation Scores Across Dropout Rates**Fig. 12** Training Time per Epoch vs Dropout Rate

The module optimization phase proved particularly insightful. Working with the established parameters (rank 16, alpha 512, dropout 0.0), the work tested various combinations of attention modules, as given in Table 17. The configuration targeting just the key and value projections [k_proj, v_proj] emerged as the optimal choice. While including additional modules like fc1 and fc2 showed marginally better metrics (0.02% higher METEOR), the [k_proj, v_proj] configuration achieved excellent scores (METEOR: 0.8296, ChrF: 0.7707), as shown in Figure 13, while reducing memory usage by 23.8% (43.25 MB vs 56.75 MB) and improving inference time by 9.75% (0.37s vs 0.41s). Despite having lower GPU memory usage (29.75 MB) and faster training time (246.52s), the [v_proj, o_proj] configuration wasn't chosen due to significant performance drops - doubled inference time (0.71s vs 0.37s), as shown in Figure 14, lower METEOR (0.7150 vs 0.8296) and ChrF scores (0.6490 vs 0.7707), making [k_proj, v_proj] the better balanced choice. Focusing LoRA adaptation on the key and value projections optimizes the core modules in transformers. By adjusting only these projections, the model efficiently learns task-specific attention patterns with fewer parameters, leading to improved performance. This also reduces the computational load during inference by minimizing parameter updates, speeding up attention calculations and resulting in faster inference without sacrificing accuracy. Table 18 mentions some translation examples noticing which the improvement can be understood better.

Table 17 Comparative analysis of LoRA fine-tuning configurations across different attention module combinations

Module	LoRA Fine-Tuned Model Additional GPU Memory (MB)	Avg. Training Time per Epoch (s)	Avg. Inference Time (s)	METEO R Score	ChrF Score	Trainable Parameters After LoRA
[q_proj, v_proj]	43.25	264.69	0.45	0.7725	0.7062	2,359,296

[q_proj, k_proj, v_proj, o_proj]	56.75	279.67	0.41	0.8279	0.7707	3,538,944
[k_proj, v_proj]	43.25	266.15	0.37	0.8296	0.7707	2,359,296
[q_proj, k_proj]	43.25	278.47	0.38	0.7978	0.7349	2,359,296
[v_proj, o_proj]	29.75	246.52	0.71	0.7150	0.6490	1,179,648
[q_proj, k_proj, v_proj, o_proj, fc1, fc2]	101.75	318.95	0.46	0.8298	0.7710	7,471,104

Table 18 Translation response seen across models fine-tuned using different target modules used

Source Text	Reference Translation	[q_proj, v_proj] Output	[q_proj, k_proj, v_proj, o_proj] Output	[k_proj, v_proj] Output	[q_proj, k_proj] Output	[v_proj, o_proj] Output	[q_proj, k_proj, v_proj, o_proj, fc1, fc2] Output	Observation
राधा ने कपड़े धोए और	Radha washed clothes	Radha washed clothes	Radha washed clothes and did the	Radha washed clothes and did the	Radha washed clothes and did the	Radha had shed clothes and done	Radha washed clothes and pressed.	The [q_proj, k_proj, v_proj, o_proj], [q_proj, v_proj], and [q_proj,

प्रेस किए।	and ironed them.	and pressed.	and pressed.	pressing.	pressing.	laundry.		k_proj, v_proj, o_proj, fc1, fc2] provide good result, the rest are incorrect.
चुटकु ले सुना ओ।	Tell jokes.	Tell a joke.	Tell a joke.	Tell jokes.	Tell jokes.	Listen quietly.	Tell a joke.	[k_proj, v_proj] and [q_proj, k_proj] are better at handling plurality, while [v_proj, o_proj] leads to an erroneous translation.
कृपया अपना उल्लू सीधा मत करो।	Please don't feather your own nest.	Please don't look out for your own benefit.	Please don't look out for your own benefit.	Please don't look out for your own benefit.	Please don't look out for your own benefit.	Please don't look around expect atively.	Please don't be selfish.	[q_proj, k_proj, v_proj, o_proj] and [q_proj, k_proj, v_proj, o_proj, fc1, fc2] perform good in handling the idiom, but [v_proj, o_proj] fail to do so.

आप तो ईद का चाँद हो गए!	You have become a rare sight!	You became the moon of Eden !	You have become the rare sight!	You have become the rare sight!	You have become the moon of Eden!	You became a fig Frost Frost Frost!	You have become a rare sight!	Best translations for [k_proj, v_proj], [q_proj, k_proj] and [q_proj, k_proj, v_proj, o_proj, fc1, fc2]; gibberish for [v_proj, o_proj]
उस पर्वता रोही को बचा लिया गया था।	The mountaineer was rescued.	He had been saved by a mountaineer.	He had been saved by a mountaineer.	The mountaineering was rescued.	The mountaineer had been rescued.	The mountaintop had been saved.	The mountaineering man had been rescued.	For best performance- [q_proj, k_proj, v_proj, o_proj].
नेताओं ने मतभेद सुलझाए।	Leaders resolved differences .	Leaders resolved the differences .	Leaders resolved the differences .	Leaders resolved the differences.	Leaders resolved the difference.	The leaders resolved differences.	Leaders resolved the differences.	All correct.

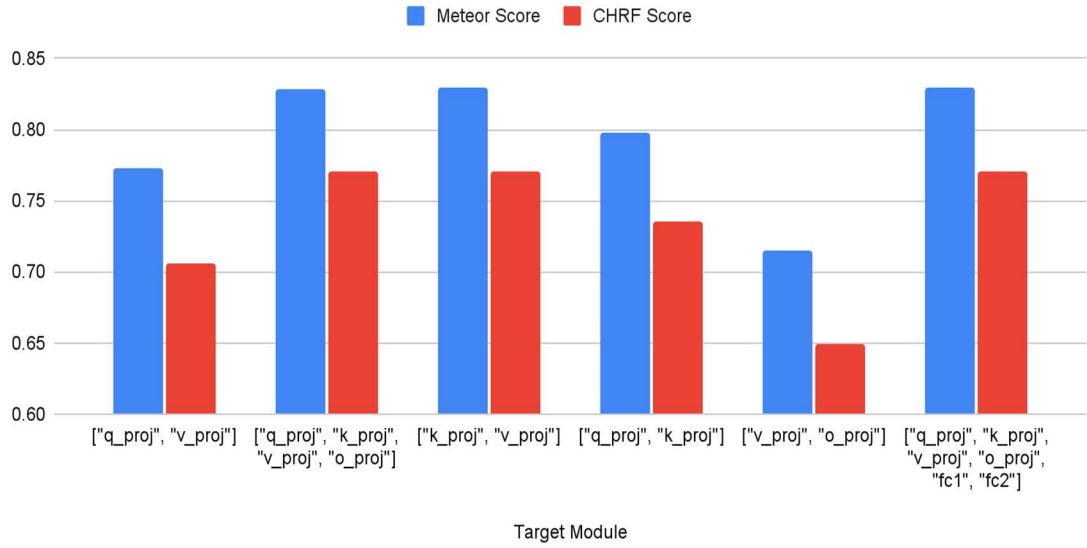


Fig. 13 Comparison of METEOR and ChrF scores across different LoRA target module configurations.

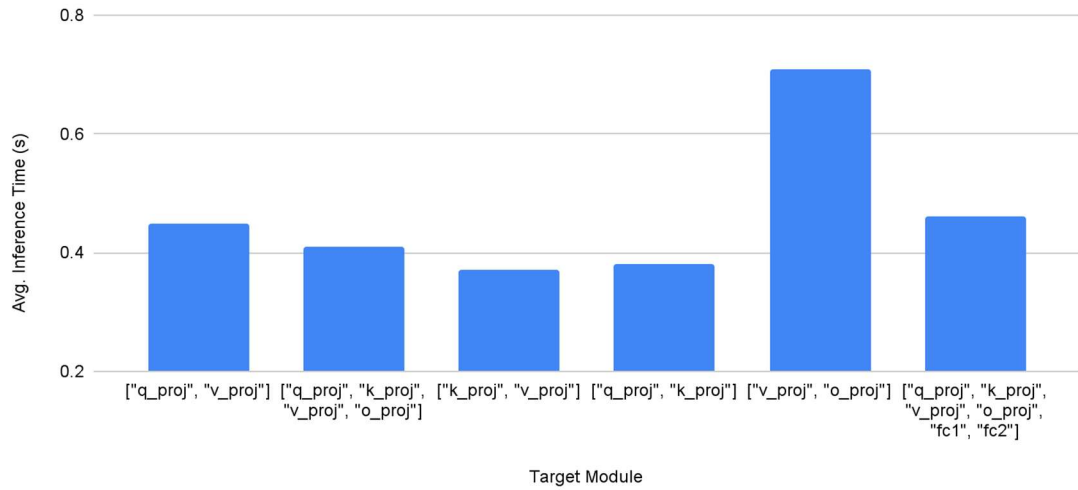


Fig. 14 Average inference time (in seconds) across different LoRA target module configurations.

4.5 Optimizing QLoRA

The QLoRA optimization study, followed a systematic approach starting with bit precision optimization. It was observed that the METEOR and ChrF scores remained stable in all the experiments of QLoRA, this may be because the configuration of LoRA giving best translation performance was already found in the previous experiments, and quantization alone will not affect the performance of the model. The selection of 4-bit quantization (over 8-bit) served as the foundation, demonstrating a compelling balance between memory efficiency and performance metrics. This initial choice informed subsequent optimizations, where the 4-bit configuration was carried forward while exploring double quantization parameters. Upon establishing False as the optimal double quantization setting, the work proceeded to optimize the compute data type, where bfloat16 emerged as superior.

Table 19 Performance Analysis of Model Training with Different Bit Quantization Settings

Bits	Quantized Model GPU Memory (MB, Megabytes)	QLoRA Fine-Tuned Model Additional GPU Memory (MB, Megabytes)	Avg. Training Time per Epoch (seconds)	METEOR Score	ChrF Score	Avg. Inference Time (seconds)
4	667.19	30.34	348.53	0.8221	0.7635	0.3785
8	2337.78	37.01	592.17	0.8208	0.7606	0.4084

We maintained consistent baseline QLoRA parameters (`double_quant = True`, `compute_dtype = float16`, `quant_type = nf4`) and incorporated the previously determined optimal LoRA configuration (`rank = 16`, `alpha = 512`, `dropout = 0.0`, `target_modules = [k_proj, v_proj]`) for the experiments. The initial investigation focused on comparing 4-bit versus 8-bit quantization, a fundamental decision that would impact all subsequent optimizations. The 4-bit configuration demonstrated remarkable efficiency, achieving a 71.4% reduction in GPU memory consumption. Performance metrics remained almost

consistent, with a 0.16% improvement in the METEOR score and 0.38% in the ChrF score. Most notably, training efficiency improved by 41.2%, while inference speed increased by 7.3%, as evident from Table 19. These impressive results can be attributed to 4-bit quantization hitting an optimal precision-memory trade-off, where the 16 distinct values (2^4) adequately preserve essential weight distributions while drastically reducing memory footprint. This compelling combination of reduced resource requirements and maintained performance metrics established 4-bit quantization as the baseline for subsequent optimizations. Using 8-bit quantization instead of 4-bit quantization increases memory requirements, inference time, and training time due to the larger bit representation for weights and activations. With 8-bit quantization, each weight requires more memory, doubling the storage compared to 4-bit. This increases the demand for memory bandwidth and reduces processing efficiency, leading to slower inference. During training, the higher precision of 8-bit weights and gradients results in more computation and memory usage per step, potentially slowing down the process.

Table 20 Memory and Performance Metrics Comparison for Double Quantization Configurations

Double Quantization	Quantized Model GPU Memory (MB, Megabytes)	QLoRA Fine-Tuned Model Additional GPU Memory (MB, Megabytes)	Avg. Training Time per Epoch (seconds)	METEOR Score	ChrF Score	Avg. Inference Time (seconds)
True	667.192871	30.338867	348.53021	0.82209	0.76345	0.3785
False	682.730957	30.338867	321.88474	0.82354	0.76362	0.3617

Building upon the 4-bit foundation, double quantization settings were evaluated. Double quantization (True) achieves better memory efficiency by applying quantization twice: first

to the model weights (reducing to 4-bit) and then to the quantization parameters themselves. In the second quantization layer, it specifically targets the scaling factors and zero points (quantization metadata), reducing them to lower precision. While this dual compression strategy showed a modest memory advantage of -2.3%, the False double quantization setting demonstrated superior operational characteristics as shown in Table 20. The faster performance with the False double quantization setting is primarily because it avoids the overhead of second-layer dequantization during both training and inference, as only one layer of quantization needs to be reversed. This elimination of additional quantization-dequantization operations led to 7.6% faster training and 4.43% faster inference. Additionally, single quantization maintained marginally better model quality resulting in improved accuracy metrics. Therefore, double quantization=False was chosen as the minimal memory benefit didn't justify the performance trade-offs.

Table 21 Evaluation of Training and Inference Performance Across Computation Data Types

Compute dtype	Avg Training Time per Epoch (seconds)	METEOR Score	ChrF Score	Avg Inference Time (seconds)
float16	321.88	0.8235	0.7636	0.3617
bfloat16	319.11	0.8234	0.7652	0.4037

With established quantization parameters (bits=4, double_quant=False), compute data type options were evaluated, results as mentioned in Table 21. In QLoRA, compute_dtype determines the precision used for intermediate computations during the training process, while keeping the base model weights in 4-bit precision. Bfloat16 emerged as the superior choice, offering small but significant improvements. METEOR scores remained almost stable, while average training time per epoch plummeted by 0.86%. Using float16 in LoRA training increases training time due to additional loss scaling and precision management. However, it reduces inference time because of faster computations on GPUs with Tensor Cores (NVIDIA T4 GPU used here) optimized for float16. While float16 provides efficient

performance during inference, bfloat16 is more stable for training due to its larger dynamic range. Even though average inference time showed an increase of 11.61%, the time in which the model is required to be adapted to recent updates, in cases such as disaster response and breaking news analysis, is more critical. This clear qualitative advantage established bfloat16 as the optimal compute data type for the configuration.

Table 22 Comparison of Memory Usage and Training Performance Metrics between Paged and Non-Paged Optimization

Metric Category	Metric	Paged	Non-Paged
GPU Memory	Peak Memory (GB, Gigabytes)	5.70	5.99
	Additional QLoRA Memory (MB, Megabytes)	14.09	43.70
Training Performance	Average Batch Time (milliseconds)	990.44	1,048.09
	Total Training Time (minutes)	28.10	29.11
	Average Time per Epoch (seconds)	343.7	357.12

Paged optimization refers to the systematic approach of managing and optimizing the paged attention mechanism. This includes tuning page sizes, managing page swapping strategies, and optimizing memory access patterns. The optimization process focuses on finding the perfect balance between memory efficiency and computational overhead.

The quantized model required 682.73 MB of memory. The paged optimization results were remarkable: peak GPU memory reduced by 4.8%, which lowers the risk of OOM (Out-Of-Memory) errors during training, and additional QLoRA memory requirements decreased by 68% (14.09MB vs 43.70MB), meaning significantly less additional memory needed during fine-tuning, as shown in Table 22. Training efficiency also improved with 5.5% faster batch processing (990.44ms vs 1,048.09ms) and 3.8% faster epoch completion

(343.70s vs 357.12s). These comprehensive improvements made paged attention a clear winner for the final configuration. The improvement in training performance may be due to the efficiency of the paging algorithm used to pre-fetch and cache data, ensuring that the GPU doesn't have to wait.

Figure 15 below shows the comparison of average training time found when using different configurations for QLoRA fine-tuning. The increase in training time for "paged optimization = True" is due to the I/O overhead and dynamic memory management costs associated with the paging mechanism. While it slightly increases training time, it enables training on larger models that wouldn't fit in GPU memory otherwise, making the trade-off worthwhile.

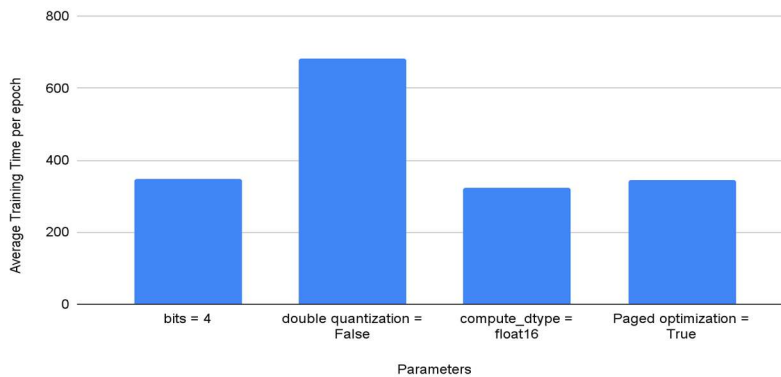


Fig. 15 Average Training Time Per Epoch (in sec) Across Various Hyperparameters Experimented for QLoRA

These results make LoRA an excellent replacement for full fine-tuning for projects with moderate computational constraints. Also, QLoRA makes a good replacement for full fine-tuned and LoRA fine-tuned models in scenarios with severe memory constraints, as it maintains nearly identical performance to full fine-tuning while using just a small amount of the resources. For scenarios where inference speed is critical, LoRA emerges as an effective alternative to full fine-tuning, offering reduced inference time and maintaining model performance. The emphasis on inference optimization is particularly crucial since model training is typically a one-time operation, while inference occurs repeatedly throughout the model's deployment lifetime. In production environments processing numerous translation requests, even modest improvements in inference speed can compound into substantial operational benefits over time. This makes LoRA particularly

attractive for real-world applications where response time is a key consideration. Also, the training time required when using QLoRA fine-tuning is 8.74% more than LoRA, hence when training time is more important, then again, the LoRA technique should be used since it maintains performance close to the QLoRA fine-tuned model.

The GPU memory requirements decrease substantially from Full Fine-Tuning to QLoRA Fine-Tuning, with LoRA presenting an intermediate memory footprint. The QLoRA Fine-Tuned model achieves comparable METEOR and ChrF scores while requiring significantly less GPU memory than the full fine-tuned and LoRA fine-tuned model. For scenarios where training speed is prioritized over memory constraints such as rapid prototyping of machine learning models, where engineers need quick feedback on model iterations, for example in a startup developing a real-time recommendation system, data scientists might need to retrain models multiple times daily to incorporate new user behavior data. Here, using LoRA would be more beneficial than QLoRA since it reduces training time by 8.74%, allowing faster model updates and quicker deployment, even though it uses more memory than QLoRA. This rapid iteration capability is especially crucial when using paid cloud GPU services, where longer training times directly translate to higher operational costs like cloud GPU rental costs, and electricity costs for running GPUs. In such cases, LoRA presents the optimal choice, as it maintains comparable performance metrics while delivering faster training times. All three fine-tuning approaches maintain comparable translation quality scores (METEOR and ChrF), despite their significantly different resource requirements.

4.6 Comparative analysis with existing similar works

Table 23 presents a comparative analysis of the proposed approach against similar works in the domain of translation systems. In recent years, optimizing translation models for speed, memory consumption, and training time has been the focus of a number of previous studies. He et al. (2021) [27] combined translation memory (TM) and neural machine translation by using one TM entry as a single bilingual sentence to accelerate training and inference. A very high BLEU was reported in this work, while being parameter efficient which makes it a good candidate for TM-specialized as well as general tasks. Liu et al. [28] decreased training time while employing both the future and the history context in a bi-LSTM using a tanh activation function, obtaining improved training efficiency but lower

BLEU scores than the baseline. In contrast, Li et al. (2020) [26] specifically on transformer architectures examined trade-offs between (1) training on vast amounts of data and (2) extreme compression approaches such as pruning and quantization that led to outstanding reductions in memory and inference times.

What is unique about this work is that it tackled challenges that are particularly relevant to low-resource settings. LoRA and the more recent QLoRA have the benefits of keeping similar translation performance as full fine-tuning while greatly reducing both GPU memory and training time compared to full fine-tuning. Inference times were also significantly more efficient than fully fine-tuned models.

Table 23 Comparison with Other Similar Works

Referen ces	Translat ion Pair	Model Used	Approach Used	Score Achieve d in Translati on Performa nce	Reduct ion in Trainin g Time seen	Reductio n in Model Memory Usage seen	Reductio n in Inferenc e Time seen
Propose d Approa ch	Hindi- English	mBART (transfor mer based model)	Full, LoRA, and QLoRA fine- tuning	Full fine- tuning- METEO R: 0.8239 ChrF: 0.7621 LoRa- METEO R: 0.8296 ChrF: 0.7707	LoRa- 70.01 % reducti on QLoR A- 61.27 % depleti on in time require	LoRa- 66.12% reductio n QLoRA- 90.06% reductio n in GPU memory usage compare d to the	LoRa- 8.16% QLoRA- 0.2% reductio n in inferenc e time compare d to the inferenc e time required

				QLoRA: METEOR-0.8234 ChrF-0.7647	d to train in contras t with the fully fine- tuned model	base model memory requirem ent	by full fine- tuned model
[27] (He, Qiuxian g et al., 2021)	Spanish -English (Es→En), English- German (En→De), Chinese -English (Zh→En)	Transfor mer with TM variants (TF-S, TF-SS, TF-SA)	Single bilingual sentence as TM, sentence similarity and alignmen t, multi- task training	BLEU Score: 68.49 (Es→En) , 28.3 (En→De)	75% reducti on in trainin g time per epoch (from 17,841 s to 4,518s)	Paramet er increase of only 8.96%	2-4x faster, from 1.91s to 0.41s per sentence
[28] (Liu et al., 2020)	German -English (De→En), English- French (En→Fr)	Bi- LSTM with tanh activatio n	Non- linear combinat ion of forward and backward context using	BLEU Score: 22.77 (De→En) , 18.52 (En→Fr)	66% reducti on in trainin g time (from 23h to 8.5h)	-	-

			tanh function				
[26] (Li et al., 2020)	English- French (En→Fr)	Transfor mer (RoBER Ta for pretraini ng, varying sizes for MT)	Training large models, then applying heavy compress ion (quantiza tion & pruning)	BLEU Score: 0.35 validatio n score with large models (2048H, 8L)	Up to 50% reducti on in trainin g time (e.g., 24- layer model trains 50% faster than 12- layer model)	60-70% memory reductio n (from quantizat ion and pruning)	2-3x faster inferenc e time due to 6- bit quantizat ion

CONCLUSION AND FUTURE WORK

In low resources scenario, where memory is at premium, this work wanted to experiment with tuning techniques LoRA and QLoRA. Detailed comparison with LoRA and QLoRA methods in terms of overall accuracy, GPU memory consumption, training time, and inference time has been performed and highlighted here. The work reported and analyzed the results from different configurations of LoRA and QLoRA. Study results showed that LoRA and QLoRA techniques are good alternatives for full fine-tuning in resource-constrained environments, as the translation performance of the model fine-tuned using these techniques was similar to the performance of full fine-tuned model, while significantly reducing resource usage.

It also provided an analysis comparing multiple training configurations for both LoRA and QLoRA, found the optimal parameter values for the model being trained on the task. The optimal combination of parameters for LoRA was able to reduce GPU memory and training time usage, requiring 66.12% less memory and 70.01% less training time, while also reducing inference time by 8.16% compared to the baseline fully fine-tuned model. With respect to the fully fine-tuned model, using QLoRA the optimal set of parameters reached a 90.06% lower GPU memory consumption, 61.27% less training time. Thus findings suggest that QLoRA is preferred for tasks where memory usage is the priority, while LoRA is better suited for scenarios where training and inference time is more critical.

For LoRA, using the rank of 16 provided best translation performance amongst other rank values experimented. This may be because the trainable parameters this rank value provides offer an effective balance between model complexity and overfitting. It was also noted that an alpha value of 512 also resulted in optimal translation performance. Furthermore, the dropout rate of 0.0 eliminated average training time per epoch, and average inference time amongst other dropout values experimented. This is because no dropout allows the model to process inputs during inference and training more quickly without the extra computational overhead that a dropout introduces during training. The results also revealed that using attention module = [k_proj, v_proj], amongst other attention modules experimented, provided the best balance between translation performance and resource efficiency. The reason for this is that in transformer architecture models, key (k_proj) and value (v_proj) projections are important for core attention mechanisms.

We also analyzed QLoRA bits quantization configurations and reported that 4-bit quantization is better than 8-bits quantization because it requires fewer computational resources. It was also found that not using double quantization led to less average training time per epoch and less average inference time because it avoids the overhead of second-layer dequantization as only one layer of quantization needs to be reversed. Even though not using double quantization required 2.3% more memory which is justified with 7.6% faster training and 11.4% faster inference time. It was also revealed that using float16 precision led to more reduction in average inference time than average training time per epoch thus this value was chosen. An experiment on quantization type was also done and found that NF4 is a better choice than FP4 because it is marginally better in terms of all the parameters experimented. NF4's lower precision and fixed-point representation require fewer bits, leading to more efficient hardware utilization and quicker computations during training and inference. QLoRA's paged optimization concept was also explored which decreased memory requirement, training time, and peak memory during training. This is because QLoRA's paged optimization dynamically manages memory, loading only necessary data into GPU memory while offloading unused parameters lowering the risk of OOM (Out-Of-Memory) errors during training.

The choice between LoRA or QLoRA largely depends on the operational context. Since training is a one-time process its effect in the delay it adds to the operation of an organization would be less as compared to inference time. For organizations that would perform frequent fine-tuning of the model to be adapted quickly to updates, in this scenario, training time would be more important and the LoRA technique is advantageous due to its reduced training time. On the other hand, in scenarios such as autonomous vehicles, healthcare diagnostics, cybersecurity, and financial systems which need to make quick predictions to act on market changes, inference time is more important than training time and QLoRA's lower memory usage make it the preferred option. Hence, LoRA or QLoRA should be chosen wisely.

Despite its contributions, this study has several limitations. Firstly, the findings are primarily based on a translation task, which may not generalize well to other NLP problems or domains where LoRA and QLoRA techniques might exhibit different performance. Furthermore, it was also subject to a certain dataset which restrict generalization on the

datasets with different linguistic or domain characteristics. The experiments were also done on a specific model architecture therefore results may vary for other architectures such as GPT or BERT which will require a different set of parameter configurations. Additionally, the study was done on NVIDIA Tesla T4 GPU and metrics- memory, training, and inference time, are hardware-dependent the results may differ. It only compared a few configurations of quantization techniques in QLoRA; it did not explore other well-studied more sophisticated strategies, e.g. mixed precision [42] or stochastic rounding [43]. Finally, although the work was primarily concerned with training on a limited number of resources, the efforts did not cover real-world deployment situations and production settings characterized by resource changes over time (e.g., due to batching or scaling) or other workload (e.g., auto-scaling, multi-tenancy) scenarios, which can impact the practical applicability of the techniques. The translation performance particularly for idiomatic sentences were not good, exploration of combination of LoRA or QLoRA and a context-sensitive technique is important. Also, the inference speed improvement was not as high as other's work, this signals future work in combining the techniques which help with inference speed improvement.

There are many future directions that can open up important paths of research in enhancing the effectiveness and applicability of the LoRA and QLoRA techniques. A nice direction to explore is tuning parameters dynamically, where auto or adaptive techniques could be used for your hyperparameters during training, so there would be no tweaking needed from a human. Noise injection could be worth investigating to increase stability in fine-tuning in light of limited resources and reduce the risk of overfitting. In addition, future work could also combine LoRA and QLoRA with other optimization algorithms such as knowledge distillation, pruning, or model ensembling for joint optimizations in resource efficiency and efficient performance. Finally, the energy cost of different LoRA and QLoRA setups will be important to assess in both single and scalable applications. Such directions could render LoRA and QLoRA even stronger and more applicable to different settings with limited resources.

REFERENCES

1. Hutchins, W. J. (1995). Machine translation: A brief history. In E. F. K. Koerner & R. E. Asher (Eds.), *Concise history of the language sciences* (pp. 431–445). Pergamon. <https://doi.org/10.1016/B978-0-08-042580-1.50066-0>
2. Rivera-Trigueros, I. (2022). Machine translation systems and quality assessment: A systematic review. *Language Resources & Evaluation*, 56, 593–619. <https://doi.org/10.1007/s10579-021-09537-5>
3. Araghi, S., Palangkaraya, A., & Webster, E. (2024). The impact of language translation quality on commerce: The example of patents. *Journal of International Business Policy*, 7, 224–246. <https://doi.org/10.1057/s42214-023-00157-0>
4. Madankar, M., Chandak, M. B., & Chavhan, N. (2016). Information retrieval system and machine translation: A review. *Procedia Computer Science*, 78, 845–850. <https://doi.org/10.1016/j.procs.2016.02.071>
5. Hudelson, P., & Chappuis, F. (2024). Using voice-to-voice machine translation to overcome language barriers in clinical communication: An exploratory study. *Journal of General Internal Medicine*, 39, 1095–1102. <https://doi.org/10.1007/s11606-024-08641-w>
6. Mohamed, Y. A., Khanan, A., Bashir, M., Mohamed, A. H. H. M., Adiel, M. A. E., & Elsadig, M. A. (2024). The impact of artificial intelligence on language translation: A review. *IEEE Access*, 12, 25553–25579. <https://doi.org/10.1109/ACCESS.2024.3366802>
7. Huynh, B. Q., Li, H., & Giger, M. L. (2016). Digital mammographic tumor classification using transfer learning from deep convolutional neural networks. *Journal of Medical Imaging*, 3(3), 034501.
8. Kermany, D. S., Goldbaum, M., Cai, W., Valentim, C. C., Liang, H., Baxter, S. L., ... & Zhang, K. (2018). Identifying medical diagnoses and treatable diseases by image-based deep learning. *Cell*, 172(5), 1122–1131.e9.
9. Vrbančič, G., & Podgorelec, V. (2020). Transfer learning with adaptive fine-tuning. *IEEE Access*, 8, 196197–196211. <https://doi.org/10.1109/ACCESS.2020.3034343>
10. Wang, Y., Yang, C., Lan, S., Zhu, L., & Zhang, Y. (2024). End-edge-cloud collaborative computing for deep learning: A comprehensive survey. *IEEE Communications Surveys & Tutorials*. <https://doi.org/10.1109/COMST.2024.3393230>
11. Raiaan, M. A. K., et al. (2024). A review on large language models: Architectures,

- applications, taxonomies, open issues and challenges. *IEEE Access*, 12, 26839–26874. <https://doi.org/10.1109/ACCESS.2024.3365742>
12. John, M. M., Olsson, H. H., & Bosch, J. (2020). AI deployment architecture: Multi-case study for key factor identification. In *Proceedings of the 27th Asia-Pacific Software Engineering Conference (APSEC)* (pp. 395–404). IEEE. <https://doi.org/10.1109/APSEC51365.2020.00048>
 13. Chen, Y., Zheng, B., Zhang, Z., Wang, Q., Shen, C., & Zhang, Q. (2021). Deep learning on mobile and embedded devices: State-of-the-art, challenges, and future directions. *ACM Computing Surveys*, 53(4), Article 84. <https://doi.org/10.1145/3398209>
 14. Sinha, S., & Lee, Y. M. (2024). Challenges with developing and deploying AI models and applications in industrial systems. *Discover Artificial Intelligence*, 4, 55. <https://doi.org/10.1007/s44163-024-00151-2>
 15. Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., ... & Chen, W. (2022). LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=nZeVKeeFYf9>
 16. Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2024). QLoRA: Efficient finetuning of quantized LLMs. In *Proceedings of the 37th International Conference on Neural Information Processing Systems (NeurIPS '23)* (pp. 10088–10115). Curran Associates Inc.
 17. Chen, J., Liu, Z., Huang, X., et al. (2024). When large language models meet personalization: Perspectives of challenges and opportunities. *World Wide Web*, 27, 42. <https://doi.org/10.1007/s11280-024-01276-1>
 18. Saxena, S., Mahajan, N., & Vinay, V. (2024). Reliability and validity of a Hindi version of the Oral Health Impact Profile (OHIP-EDENT-H) for edentulous subjects: A cross-sectional study. *Journal of Oral Biology and Craniofacial Research*, 14(6), 676–681. <https://doi.org/10.1016/j.jobcr.2024.09.005>
 19. Singh, M., Kumar, R., & Chana, I. (2021). Machine translation systems for Indian languages: Review of modelling techniques, challenges, open issues and future research directions. *Archives of Computational Methods in Engineering*, 28, 2165–2193. <https://doi.org/10.1007/s11831-020-09449-7>
 20. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... &

- Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems* (Vol. 30). Curran Associates Inc.
21. Lin, D., Wen, Y., Wang, W., & Su, Y. (2024). Enhanced sentiment intensity regression through LoRA fine-tuning on Llama 3. *IEEE Access*, 12, 108072–108087. <https://doi.org/10.1109/ACCESS.2024.3438353>
 22. Ghosh, A., et al. (2024). MedSumm: A multimodal approach to summarizing code-mixed Hindi-English clinical queries. In N. Goharian et al. (Eds.), *Advances in information retrieval. ECIR 2024. Lecture Notes in Computer Science* (Vol. 14612). Springer. https://doi.org/10.1007/978-3-031-56069-9_8
 23. Liu, Y., Gu, J., Goyal, N., Li, X., Edunov, S., Ghazvininejad, M., Lewis, M., & Zettlemoyer, L. (2020). Multilingual denoising pre-training for neural machine translation. *Transactions of the Association for Computational Linguistics*, 8, 726–742. https://doi.org/10.1162/tacl_a_00343
 24. Xue, L., Constant, N., Roberts, A., Kale, M., Al-Rfou, R., Siddhant, A., Barua, S., & Raffel, C. (2021). mT5: A massively multilingual pre-trained text-to-text transformer. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (pp. 483–498). <https://doi.org/10.18653/v1/2021.naacl-main.41>
 25. Ou, Y., Hui, Z., Zhou, T., Cai, Y., & Li, J. (2024). Llama2-13b-based NEFT fine-tuning for financial sentiment classification. In *Proceedings of the 2024 Guangdong-Hong Kong-Macao Greater Bay Area International Conference on Digital Economy and Artificial Intelligence (DEAI '24)* (pp. 641–644). ACM. <https://doi.org/10.1145/3675417.3675523>
 26. Li, Z., Wallace, E., Shen, S., Lin, K., Keutzer, K., Klein, D., & Gonzalez, J. E. (2020). Train large, then compress: Rethinking model size for efficient training and inference of transformers. In *Proceedings of the 37th International Conference on Machine Learning (ICML'20)* (Vol. 119, pp. 5958–5968). JMLR.
 27. He, Q., et al. (2021). Fast and accurate neural machine translation with translation memory. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*.
 28. Liu, X., Wang, W., Liang, W., et al. (2020). Speed up the training of neural machine

translation. *Neural Processing Letters*, 51, 231–249. <https://doi.org/10.1007/s11063-019-10084-y>

29. Tuli, S., & Jha, N. K. (2024). EdgeTran: Device-aware co-search of transformers for efficient inference on mobile edge platforms. *IEEE Transactions on Mobile Computing*, 23(6), 7012–7029. <https://doi.org/10.1109/TMC.2023.3328287>
30. Park, Y., Budhathoki, K., Chen, L., Kübler, J. M., Huang, J., Kleindessner, M., Huan, J., Cevher, V., Wang, Y., & Karypis, G. (2024). Inference optimization of foundation models on AI accelerators. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD '24)* (pp. 6605–6615). ACM. <https://doi.org/10.1145/3637528.3671465>
31. Zhu, Y., et al. (2024). MeLo: Low-rank adaptation is better than fine-tuning for medical image diagnosis. In *2024 IEEE International Symposium on Biomedical Imaging (ISBI)* (pp. 1–5). <https://doi.org/10.1109/ISBI56570.2024.10635615>
32. Zhang, Q., et al. (2024). Exploring edge-driven collaborative fine-tuning towards customized AIGC services. *IEEE Network*. <https://doi.org/10.1109/MNET.2024.3441040>
33. Agiza, M. N., & Reda, S. (2024). MTLORA: A low-rank adaptation approach for efficient multi-task learning. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 16196–16205). <https://doi.org/10.1109/CVPR52733.2024.01533>
34. Fu, J., Ge, X., Xin, X., Karatzoglou, A., Arapakis, I., Wang, J., & Jose, J. M. (2024). IISAN: Efficiently adapting multimodal representation for sequential recommendation with decoupled PEFT. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)* (pp. 687–697). <https://doi.org/10.1145/3626772.3657725>
35. Chakraborty, P., Arumugam, K. K., Alfadel, M., Nagappan, M., & McIntosh, S. (2024). Revisiting the performance of deep learning-based vulnerability detection on realistic datasets. *IEEE Transactions on Software Engineering*, 50(8), 2163–2177. <https://doi.org/10.1109/TSE.2024.3423712>
36. Maatouk, N., Piovesan, F., Ayed, A., Di Domenico, A., & Debbah, M. (2024). Large language models for telecom: Forthcoming impact on the industry. *IEEE Communications Magazine*. <https://doi.org/10.1109/MCOM.001.2300473>

37. Li, S., & Huang, Y. (2024). BAS-ALSTM: Analyzing the efficiency of artificial intelligence-based English translation system. *Journal of Ambient Intelligence and Humanized Computing*, 15, 765–777. <https://doi.org/10.1007/s12652-023-04735-1>
38. Bhardwaj, S., Singh, P., & Pandit, M. K. (2024). A survey on the integration and optimization of large language models in edge computing environments. In *2024 16th International Conference on Computer and Automation Engineering (ICCAE)* (pp. 168–172). <https://doi.org/10.1109/ICCAE59995.2024.10569285>
39. Yu, B. X. B., Chang, J., Wang, H., Liu, L., Wang, S., Wang, Z., Lin, J., Xie, L., Li, H., Lin, Z., Tian, Q., & Chen, C. W. (2024). Visual tuning. *ACM Computing Surveys*, 56(12), Article 297, 1–38. <https://doi.org/10.1145/3657632>
40. Tang, Y., et al. (2020). Multilingual translation with extensible multilingual pretraining and finetuning. *arXiv preprint*. <https://arxiv.org/abs/2008.00401>
41. Lewis, M., Liu, Y., Goyal, N., Ghazvininejad, M., Mohamed, A., Levy, O., Stoyanov, V., & Zettlemoyer, L. (2020). BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 7871–7880). Association for Computational Linguistics. <https://aclanthology.org/2020.acl-main.703>
42. Rakka, M., Fouda, M. E., Khargonekar, P., & Kurdahi, F. (2024). A review of state-of-the-art mixed-precision neural network frameworks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 46(12), 7793–7812. <https://doi.org/10.1109/TPAMI.2024.3394390>
43. Jeong, S., Choi, D., & Kim, H. (2024). SRU-Q: Hardware-friendly stochastic rounding unit-based gradient quantization for CNN training. In *2024 IEEE 6th International Conference on AI Circuits and Systems (AICAS)* (pp. 457–461). <https://doi.org/10.1109/AICAS59952.2024.10595985>

APPENDIX A

PAPER ACCEPTANCE MAIL

12/6/24, 8:11 PM

Netaji Subhas University of Technology Mail - ICCIS 2024: Notification of your paper ID 304: Acceptance



DAKSH . <daksh.ug21@nsut.ac.in>

ICCIS 2024: Notification of your paper ID 304: Acceptance

1 message

Microsoft CMT <email@msr-cmt.org>
Reply-To: ICCIS SCRS <scrs.iccis@gmail.com>
To: Daksh Kalucha <daksh.ug21@nsut.ac.in>

Wed, Oct 9, 2024 at 11:43 PM

Dear Daksh Kalucha,

Greetings!

Thank you for submitting your research article to the 6th International Conference on Communication and Intelligent Systems (ICCIS 2024) to be held on November 08-09, 2024 at Maulana Azad National Institute of Technology (MANIT), Bhopal, India in Hybrid Mode.

We are pleased to inform you that based on reviewers' comments, your paper titled "Enhancing Pre-trained Models for Hindi-English Translation: A Multi-faceted Fine-tuning Approach" has been accepted for presentation during ICCIS 2024, and publication in the proceedings to be published in Scopus-indexed Springer Book Series "Lecture Notes in Networks and Systems" subject to the condition that you submit a revised version as per the comments, available at Authors CMT account. It is also required that you prepare a response to each comment from the reviewer and upload it as a separate file along with the revised paper.

The similarity index in the final paper must be less than 20%. Please note that the high plagiarism and any kind of multiple submissions of this paper to other conferences or journals will lead to rejection at any stage. Please note that the publisher, i.e. Springer Nature may ask for any other changes during the production which are supposed to be implemented. The publisher has the final right to exclude the paper from the proceedings if they found it unsuitable for publication.

Please carry out the steps to submit the camera-ready paper and online registration (Under "Regular Author" Category) as per the instructions available at

https://scrs.in/conference/iccis2024/page/Camera_Ready_Paper_Submission

In order to register in the SCRS member category (subsidized registration fees), you can first become a member at <https://www.scrs.in/register> and then register for the conference OR you may register as a Regular Author Category.

Please note that the Last date for submission of the camera-ready paper, payment of the registration fee, and online registration is October 24, 2024.

Feel free to write to the "General Chairs, ICCIS 2024" at scrs.iccis@gmail.com, should you have any questions or concerns. Please remember to always include your Paper ID- 304, whenever inquiring about your paper.

Looking forward to meeting you during the conference.

With Regards

Team ICCIS 2024 (<https://scrs.in/conference/iccis2024>)
Maulana Azad National Institute of Technology (MANIT), Bhopal, India

To stop receiving conference emails, you can check the 'Do not send me conference email' box from your User Profile.

Microsoft respects your privacy. To learn more, please read our [Privacy Statement](#).

Microsoft Corporation
One Microsoft Way
Redmond, WA 98052

<https://mail.google.com/mail/u/0/?ik=a5b51155d2&view=pt&search=all&permthid=thread-f:1812461144911810943&simpl=msg-f:1812461144911810943> 1/

APPENDIX B

CERTIFICATE OF PRESENTATION AND PAPER PUBLISHING STATUS

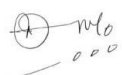


6th International Conference on Communication and Intelligent Systems (ICCIS 2024)
Organized by
Maulana Azad National Institute of Technology (MANIT), Bhopal, India
Technically Sponsored by
Soft Computing Research Society
November 08-09, 2024



Certificate of Presentation

This is to certify that **Daksh Kalucha** has presented the paper titled **Enhancing Pre-Trained Models for Hindi-English Translation: A Multi-Faceted Fine-Tuning Approach** authored by **Daksh Kalucha, Sachit Gupta, Prakhar Verma, Bharti Nagpal, Shobha Bhatt** in the 6th International Conference on Communication and Intelligent Systems (ICCIS 2024) held during November 08-09, 2024.


Prof. Sanjay Sharma
(General Chair)


Dr. Harish Sharma
(General Chair)

<https://scrs.in/conference/iccis2024>



DAKSH . <daksh.ug21@nsut.ac.in>

Call for Papers: SCRS sponsored conferences

1 message

Microsoft CMT <email@msr-cmt.org>
Reply-To: ICCIS SCRS <scrs.iccis@gmail.com>
To: Daksh Kalucha <daksh.ug21@nsut.ac.in>

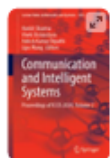
Tue, Mar 25, 2025 at 8:22 PM

Dear Daksh Kalucha,

Greetings from Soft Computing Research Society!

Thank you for your submission/participation in SCRS sponsored conference 6th International Conference on Communication and Intelligent Systems as an author/reviewer/Committee member. Proceedings of ICDSA is scheduled to be published by June/July 2025. However, it may be extended depending on response by authors for proofread.

Home > Conference proceedings



Communication and Intelligent Systems

Proceedings of ICCIS 2024, Volume 2

| Conference proceedings | Jun 2025

Overview

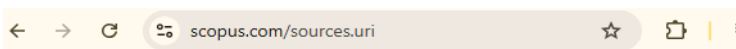
Editors: [Harish Sharma](#), [Vivek Shrivastava](#), [Ashish Kumar Tripathi](#), [Lipo Wang](#)

Presents research works in the field of communication and intelligent systems

Includes original works presented at ICCIS 2024 held in Bhopal, India

Provides state of the art to research scholars and postgraduates from various domains of engineering and related fields

Part of the bookseries: [Lecture Notes in Networks and Systems](#) (LNNS, volume 1372)



Scopus 20

Empowering discovery since 2004

Sources

ISSN



Enter ISSN or ISSNs

ISSN: 2367-3389 x

Filter refine list

Apply

Clear filters

Display options

☐ Display only Open Access journals

Counts for 4-year timeframe

1 result

☐ All

Export to Excel

Save

Source title

☐ 1 Lecture Notes in Networks and Systems

APPENDIX C
CONFERENCE PAPER WORK
AND EXTRA WORK DONE

Original (Hindi)	Layer-wise FT Translation	Discriminative FT Translation	Hybrid (GU + DFT) Translation	Analysis
वह यूजर अकाउंट्स को मैनेज कर रहा है।	He is managed to user accounts.	He is analyzing the user accounts.	He is managing the user accounts.	Hybrid is most accurate. Base is grammatically broken.
अक्सर अपना उल्लू सीधा करने वाले लोग विश्वासघात करते हैं।	Most of the people who look out for own benefit are persons.	Often people who look out for their own benefit betray themselves.	Often people who look out for their own benefit betray trust.	Hybrid gives a better English rendering; fine- tuned is acceptable. Base is broken.
उसे ऊँची दुकान फीका पकवान मिला!	He had bought very show!	He found the high shop all the time!	He found all that was all show and no substance!	Hybrid captures idiom perfectly. Both earlier versions fail.
कृपया दूर के ढोल सुहावने मत समझो।	Please don't understand the grass on the other side.	Please don't think the grass is greener on the other side.	Please don't judge the grass on the other side.	Fine-tuned gives the standard idiom. Hybrid is close but less idiomatic.

योग करो।	Do admise.	Do yoga.	Pay attention.	Only fine-tuned gets it right. Hybrid misinterprets. Base is incoherent.
टहलने जाओ।	Go away.	Go a walk.	Go on a walk.	Hybrid corrects fine-tuned grammar.

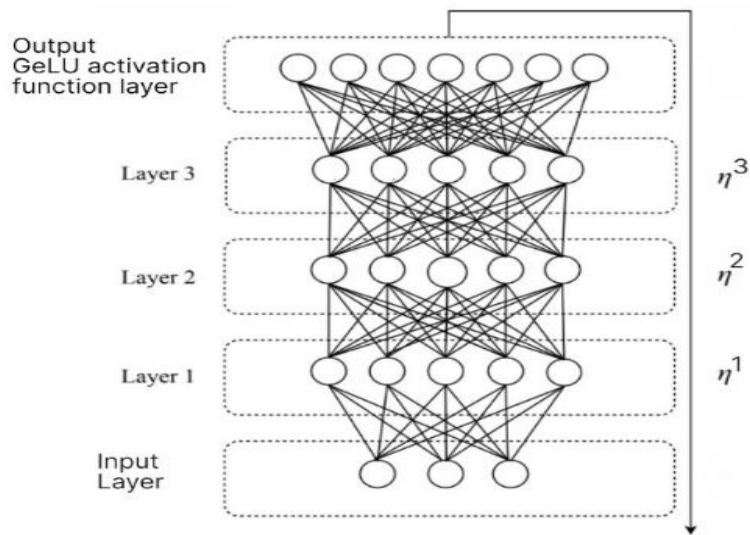


Fig. 1A Architecture depicting the discriminative fine-tuning model

APPENDIX D

PLAGIARISM REPORT



Page 2 of 49 - Integrity Overview

Submission ID trn:oid::27535:95330140

5% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Small Matches (less than 8 words)

Match Groups

- 59 Not Cited or Quoted 4%**
Matches with neither in-text citation nor quotation marks
- 8 Missing Quotations 1%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 4% Internet sources
- 3% Publications
- 4% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

No suspicious text manipulations found.

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.



Page 2 of 49 - Integrity Overview

Submission ID trn:oid::27535:95330140

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups



0 AI-generated only 0%

Likely AI-generated text from a large-language model.



0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.

